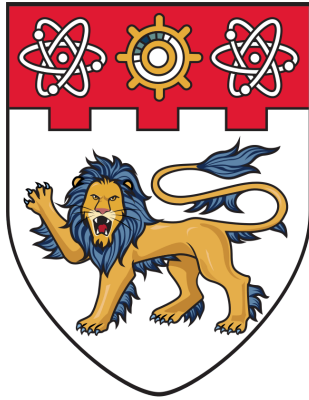




**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

**Moving-Horizon-Estimation-Enabled
Nonlinear Model Predictive Control
for Quadrotors under Abrupt Payload
Changes**

CANDIDATE NAME

SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING

2026

Moving-Horizon-Estimation-Enabled Nonlinear Model Predictive Control for Quadrotors under Abrupt Payload Changes

CANDIDATE NAME

SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING

**A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE IN [PROGRAMME]**

2026

Statement of Originality

I hereby certify that the work embodied in this thesis is the result of original research and has not been submitted for a higher degree to any other University or Institution.

.....

Date

.....

CANDIDATE NAME

Supervisor Declaration Statement

I have reviewed the content and presentation style of this thesis and declare it is free of plagiarism and of sufficient grammatical clarity to be examined. To the best of my knowledge, the research and writing are those of the candidate except as acknowledged in the Author Attribution Statement. I confirm that the investigations were conducted in accord with the ethics policies and integrity standards of Nanyang Technological University and that the research data are presented honestly and without prejudice

.....
Date

.....
Supervisor's Name

Authorship Attribution Statement

This thesis does not contain any materials from papers published in peer-reviewed journals or from papers accepted at conferences in which I am listed as an author.

.....

Date

.....

CANDIDATE NAME

Table of Contents

Abstract	v
Acknowledgements	viii
Acronyms	ix
Symbols	x
Lists of Figures	xiii
Lists of Tables	xiv
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Research Objectives	3
1.4 Scope	4
1.5 Contributions	5
1.6 Dissertation Organisation	6
2 Background and Related Work	8
2.1 Model Predictive Control	8
2.2 Quadrotor NMPC	9
2.3 Uncertainty and Adaptation in MPC	9
2.4 State and Parameter Estimation	10
2.5 Inertial-Parameter Identification	11
2.6 Change Detection and Event-Triggered Estimation	11
2.7 Research Gap	12
3 Modeling and Parameter Structure	14
3.1 Frames, Notation, and Conventions	15
3.1.1 Attitude representation	16
3.2 Rigid-Body Kinematics	16
3.3 Rigid-Body Dynamics	18
3.3.1 Translational dynamics	18
3.3.2 Rotational dynamics	19
3.3.3 The thrust eccentricity moment	20

3.3.4	Structure of the parameter entry points	21
3.4	Payload-Induced Parameters	21
3.4.1	Centre-of-mass offset	22
3.4.2	Inertia increment and the reduced mass	22
3.4.3	Magnitudes on the experimental platform	24
3.5	The First Mass Moment as the Payload State	25
3.5.1	Why the offset is the wrong coordinate	25
3.5.2	The two channels the first moment drives	26
3.5.3	Re-expressing the inertia increment	27
3.5.4	The inertia lever and the frozen coefficient	29
3.5.5	Smooth positive part	30
3.6	Model-Independent Reconstruction of the Physical Input	31
3.7	Estimation: Mass and First Moment as the Online Degrees of Freedom	33
3.7.1	Augmented state and estimator formulation	33
3.7.2	Which residual identifies what	34
3.7.3	Prior-free initialisation	36
3.8	Control: How the Parameters Re-centre the Cost	37
3.8.1	Why the input baseline must track the parameters	38
3.8.2	Inner-loop bandwidth	39
3.9	What is Estimated, What is Derived, and What is Not	40
3.10	Declared Modelling Approximations	42
4	The Eventless MHE–NMPC Framework	44
4.1	Closed-Loop Architecture	44
4.2	Moving Horizon Estimator	46
4.2.1	The optimisation	46
4.2.2	Weights	47
4.2.3	Stage weighting: what was tried and what is deployed	49
4.2.4	Re-anchoring and over-range rejection	50
4.3	The Continuous Payload Interface	51
4.3.1	One atomic frame	51
4.3.2	Filtering and slew limiting	52
4.4	The Eventless Grasp–Release Lifecycle	53
4.4.1	Why refuse the command	53
4.4.2	Two state machines	53
4.4.3	No-payload confidence	55
4.4.4	The loaded moment reference	56
4.4.5	Release evidence and the decision rule	57
4.4.6	The release-residual vote	59
4.4.7	Confirmation, UNRESOLVED, and braking to hover	60
4.4.8	Threshold provenance	61
4.5	Adaptive NMPC	62
4.5.1	Cost and weights	62
4.5.2	Solver	64
4.6	Inner-Loop Scheduling	64

4.6.1	From inertia ratio to a command scale	64
4.6.2	Headroom limiting	65
4.6.3	Attach-time inertia bootstrap	66
4.7	Reference Trajectory	67
4.8	Online Execution Sequence	68
5	Implementation and Experimental Evaluation	70
5.1	Software-in-the-Loop Platform	70
5.2	Platform Parameters	71
5.3	Experiment Design	72
5.3.1	Release-baseline arms	72
5.3.2	Conditions and scenarios	73
5.4	Metrics	74
5.5	Real-Time Performance	75
5.6	Estimation of the Payload States	75
5.7	Completion and Confirmation Delay	76
5.8	Does Refusing the Command Buy Anything?	79
5.8.1	Nominal pairs	79
5.8.2	Delayed detachment	80
5.8.3	Command-armed confirmation and unsignaled loss	80
5.9	Partial Payload Loss	81
5.10	Ablations	82
5.10.1	Thrust-map calibration	82
5.10.2	Window-balance re-initialisation	83
5.10.3	The inertia-coefficient mode	83
5.10.4	A negative result on learned stage weights	84
5.11	Where the Interface Fails	84
5.11.1	Moment-reference formation	84
5.11.2	First-moment drift under sustained excitation	85
5.11.3	The operating limit	85
5.11.4	Estimator failures	86
5.11.5	Configuration and default-safety	86
5.11.6	Missing baselines	86
5.12	Threats to Validity and Reproducibility	87
5.13	Development-Stage Evidence	88
6	Conclusion and Future Work	89
6.1	Summary	89
6.2	Principal Findings	91
6.3	Limitations	92
6.4	Future Work	93
6.5	Closing Remark	95
	References	96
	Appendix A Implemented Optimal Control Problem Configuration	101

Appendix B	Experiment Record Template	104
Appendix C	Pre-Submission Checklist	106

Abstract

Delivery quadrotors experience abrupt and coupled changes in mass, inertia and centre of mass when a payload is grasped or released. A nonlinear model predictive controller (NMPC) that retains the unloaded model applies an incorrect hover thrust, underestimates the moment demanded by an eccentric load, and loses the intended constraint margin. This dissertation develops a moving-horizon-estimation-enabled NMPC architecture that adapts the predictive model online from ordinary flight signals alone, and — the central claim — that never treats an attach or release command as evidence that the payload has changed.

The controller issues the gripper release command itself and still does not use it to clear its payload model. The motivation is that a release command is a request while the model must describe the vehicle: a gripper can be slow, can jam, can release part of a load, and a load can be lost with no command at all. A command-triggered controller then flies a loaded vehicle with an empty model, silently.

Making that refusal affordable requires a payload parameterisation that remains well posed through release. The conventional pair — payload mass and grasp offset — is singular in the limit that matters: as the payload mass tends to zero the offset becomes undefined, and an estimator holding it fixed can cancel the resulting phantom trim moment only by corrupting its mass estimate, which was observed as 14 of 55 post-release samples resting on the mass lower bound. The estimator developed here augments the composite mass together with the horizontal *first mass moment* $s = mpr_{xy}$, the product that the equations of motion actually contain, giving a 16-state moving horizon estimator over a 2.0 s

window at 10 Hz. The centre-of-mass offset then follows as $c = s/m$ with no division by the payload mass and no knowledge of the grasp geometry; the off-diagonal inertia terms follow with the payload mass cancelling exactly; and the dominant roll/pitch increment follows from the composite mass and a weak vertical-arm prior. Two reductions are derived and quantified: the second-order horizontal inertia terms carry a factor $1/m_P$ in these coordinates and are discarded for a 2.6% bias, and the remaining mass-dependent inertia coefficient, which otherwise lets the optimiser dilute a phantom moment by inflating the mass, is frozen within each window.

The physical wrench driving the estimator — collective thrust, roll and pitch moments, and the yaw reaction — is reconstructed from rotor speeds, so the estimator is never driven by a signal computed from its own output. A window vertical-force balance supplies a prior-free re-initialisation that uses only the two velocity endpoints of the window. One atomic, health-gated estimate frame feeds both the 13-state NMPC (20 stages at 0.05 s, 20 Hz) and a continuous body-rate command scaling with explicit headroom limiting. The lifecycle is split between two state machines: the estimator owns payload presence and may declare release on a collapse of the first moment relative to a self-formed, frozen reference; the controller owns the model and clears it only on persistent confidence. An unconfirmed release becomes UNRESOLVED — a hold with a smooth brake to hover — never an assumption of completion.

The system is implemented in ROS 2 around PX4 software-in-the-loop, Gazebo and acados. In the frozen campaign the interface confirmed release in 32 of 36 eligible flights through three confirmation paths, with median delays of 3.46 s in manoeuvre and 16.56 s after braking to hover. Under a 4 s delayed detachment a command-triggered baseline cleared its model early in 5 of 5 pairs while the proposed interface did so in none of six runs. Against a variant armed only by the command, eventlessness detected 10 of 18 unsignaled payload losses that the command-armed variant cannot see, with no false declaration in 209 loaded flights. The costs are reported as plainly as the benefits: confirmation is delayed

and the delay is bimodal, roughly 40 % of flights never form the moment reference the detector needs, and a ± 5 % error in the estimator's thrust coefficient defeats payload recognition in both directions.

Keywords: quadrotor; nonlinear model predictive control; moving horizon estimation; first mass moment; inertial parameter identification; eventless payload adaptation; autonomous release detection; acados; PX4 SITL.

Acknowledgements

I would like to thank my supervisor, **[SUPERVISOR NAME]**, for guidance throughout this dissertation. I am also grateful to the members of **[LABORATORY OR RESEARCH GROUP]** for their technical discussions and support with the simulation and flight-control platform.

[DRAFT NOTE: Personalise this section and add funding, laboratory and collaborator acknowledgements where applicable.]

Acronyms

Acronym	Meaning
CoM	Centre of Mass
CRLB	Cramér–Rao Lower Bound
CUSUM	Cumulative Sum (change-detection procedure)
EKF	Extended Kalman Filter
ENU	East–North–Up
ESC	Electronic Speed Controller
FLU	Front–Left–Up
FSM	Finite State Machine
INDI	Incremental Nonlinear Dynamic Inversion
IQR	Interquartile Range
LPF	Low-Pass Filter
MAD	Median Absolute Deviation
MHE	Moving Horizon Estimation
MPC	Model Predictive Control
NLP	Nonlinear Programming
NMPC	Nonlinear Model Predictive Control
OCF	Optimal Control Problem
PX4	PX4 autopilot software stack
QP	Quadratic Programming
RK4	Fourth-order Runge–Kutta integration
RMS(E)	Root Mean Square (Error)
ROS	Robot Operating System
RTI	Real-Time Iteration
SITL	Software-in-the-Loop
SQP	Sequential Quadratic Programming
UAV	Unmanned Aerial Vehicle
ZOH	Zero-Order Hold

Symbols

Symbol	Definition
<i>States and inputs</i>	
$\mathbf{p}, \mathbf{v}$	Position and velocity in the world frame (ENU)
$\mathbf{q}$	Hamilton unit quaternion, scalar component first
$\boldsymbol{\omega}$	Body angular velocity (FLU body frame)
$T, \boldsymbol{\tau}$	Collective thrust and body torque
θ	Tilt angle of the body z axis from the vertical
<i>Payload parameters</i>	
m_B, m_P, m_T	Bare-airframe, payload and composite mass
$\mathbf{r}_p = [r_x, r_y, r_z]$	Payload attachment point in the body frame
$\mathbf{r}_{xy} = [r_x, r_y]$	Horizontal part of the attachment point
$\mathbf{c} = [c_x, c_y, c_z]$	Composite centre of mass in the body frame, $\mathbf{c} = (m_P/m_T)\mathbf{r}_p$
$\mathbf{c}_{xy} = [c_x, c_y]$	Horizontal part of the composite centre of mass, $\mathbf{c}_{xy} = \mathbf{s}/m_T$
$\mathbf{s} = [s_x, s_y]$	Horizontal first mass moment, $\mathbf{s} = m_P\mathbf{r}_{xy}$
μ	Reduced mass, $\mu = m_B m_P / m_T$
J_B, J_T	Bare-airframe and composite inertia tensors
ΔJ	Payload-induced roll/pitch inertia increment
A, B_i, C_{ij}	Inertia-increment groups, (3.28)
$\boldsymbol{\tau}_{\text{com}}$	Thrust eccentricity moment about the composite CoM
κ	Inertia ratio $(J_{xx} + \Delta J)/J_{xx}$
<i>Reconstructed input and actuator model</i>	
$T_{\text{phys}}, \boldsymbol{\tau}_{\text{phys}}$	Wrench reconstructed from rotor speeds
k_f, k_m	Rotor thrust and yaw-reaction coefficients
η	Normalised throttle command
k_d, g	Linear drag coefficient, gravitational acceleration
<i>Estimator and controller</i>	
$N, \Delta t$	Horizon length and sampling interval
W_y, W_w, W_0	MHE measurement, process-noise and arrival weights
β_j	Per-stage weighting factor; λ forgetting factor
Q, R, P	NMPC stage-state, input and terminal weights
$L(\cdot)$	Bryson tolerance scale for the corresponding residual
u_{hover}	Parameter-dependent equilibrium input baseline
$\mathbf{d}, \boldsymbol{\xi}$	Lumped translational and rotational disturbances (baseline only)
<i>Lifecycle</i>	
γ	No-payload confidence, (4.11)
$\sigma(v; a, b)$	Cubic smooth-step indicator, (4.10)
ρ	Moment-collapse ratio $\ \mathbf{s}\ /\ \mathbf{s}\ _{\text{ref}}$
$\ \mathbf{s}\ _{\text{ref}}$	Frozen loaded first-moment reference
ΔT	Half-window thrust residual, (4.18)
ς	Body-rate command scale

List of Figures

4.1	System architecture and information flow. The complete pipeline consumes only rotor speeds and odometry. One atomic, health-gated $m/s/J$ frame drives both the NMPC model and the continuous body-rate command scheduling. No attach or release notification enters this path.	45
4.2	The payload lifecycle as two state machines. The estimator owns payload presence and may declare release from the evidence of (4.15); the controller owns the model and clears it only on confirmation, $\gamma > 0.90$ held for 1.0 s. The gripper release command enters neither machine. UNRESOLVED is a hold, not a clearing: the model stays loaded while the vehicle brakes to hover and keeps waiting for evidence.	54
5.1	First mass moment against ground truth (0.15 kg at 2 ms^{-1} , $n = 11$, each flight normalised by its own truth). Truth is reconstructed offline from the simulator-measured grasp offset and never reaches the estimator. The carry-phase estimate is biased toward zero in the batch shown; the cause is not identified. The moment-gated detector uses the ratio ρ of (4.12), which is insensitive to a common scale bias.	76
5.2	Measured mission trajectory (simulator ground truth), two $0.15 \text{ kg}/2 \text{ ms}^{-1}$ flights of the same batch: grasp, lift, then carry along the figure-eight with $\pm 0.8 \text{ m}$ vertical undulation against the nominal reference. After the release command one flight clears the moment vote in manoeuvre and keeps tracking; the other declares UNRESOLVED at its 12 s timeout and brakes to hover, confirming at 16.40 s. The two paths agree to within 0.12 m (median) before the timeout. Loaded tracking RMS is 0.053 m and 0.060 m after release.	77
5.3	One eventless flight from the final batch (0.15 kg , 2 ms^{-1}). Left: measured horizontal path, coloured by carrying and post-release phase, against the nominal reference including its amplitude ramp. Right: height, the mass estimate actually consumed by the controller, and the position-error norm through grasp, lift, carry, release command and confirmation. This flight confirms in manoeuvre 4.46 s after the command; it illustrates one execution and does not replace the cohort statistics of Table 5.3.	78

List of Tables

3.1	Treatments of the inertia coefficient A . Breaking the derivative removes the gradient path of (3.33); retaining a correct value is a separate requirement, which the last two forms fail.	30
3.2	Model parameters, classified by identity. Only the composite mass and the two components of the horizontal first mass moment are decision variables of the estimator; the inertia increment and the CoM offset are generated from them.	41
4.1	Lifecycle thresholds and their provenance. “Earlier flights” pre-date, and are disjoint from, the reported campaign.	62
5.1	Main implementation components of the deployed pipeline.	71
5.2	Release baselines implemented as switchable arms. The estimator and the controller each implement their half of the same named setting.	73
5.3	Release completion and confirmation delay, all eligible arm B flights. Delay runs from the release command to the end of the 1.0s confidence hold and is computed over completed flights. The two delay columns are distinct control paths and are not pooled.	78
5.4	Confirmation paths in the 32 completed flights of Table 5.3. <i>Moment-gated</i> : the detector declared before the confidence crossed. <i>Mass-domain</i> : the frozen reference never formed, the confidence crossed during or shortly after the brake to hover, and the mass-domain route declared 0.9s to 3.1s later. <i>Confidence only</i> : the reference never formed and no declaration occurred.	79
5.5	Paired command-triggered ablation. Arm A clears the model at the command by construction; arm B requires estimator evidence; arm C is arm B with its release evidence armed only by the command. The last two rows inject a command/physical-detachment mismatch; “early” means model clearing before the joint is actually removed (“–”: not scored).	79
5.6	Attempts and exclusions in the paired campaign. Because the arms differ before release, these rates are reported per arm rather than pooled, and neither characterises end-to-end mission reliability.	88

A.1	NMPC configuration (deployed default).	101
A.2	MHE configuration (deployed default). The prior-free column lists the values taken by the ablation of Section 4.2.2.	102
A.3	Payload interface and lifecycle settings.	103

Chapter 1

Introduction

1.1 Background

Quadrotors are increasingly used for inspection, transport and delivery because they can hover and manoeuvre in confined three-dimensional environments. Their agility, however, is obtained from a light airframe with limited thrust and torque margins. A package pickup can therefore change the vehicle dynamics on the same time scale as the control loop. The total mass changes the thrust-to-acceleration map, an offset load produces a persistent moment about the composite centre of mass, and the payload lever arm changes the rotational inertia.

Model predictive control is attractive for this problem because it predicts future motion while handling actuator limits and coupled dynamics explicitly [1, 2]. For a quadrotor the relevant equations are nonlinear and the attitude evolves on the unit-quaternion manifold; nonlinear MPC therefore avoids the loss of fidelity caused by repeated linearisation over aggressive motion [3, 4]. Its performance nevertheless depends on the model supplied to the prediction. If the controller continues to use the empty-airframe parameters after grasping a parcel, even a numerically converged optimisation solves the wrong problem.

This dissertation couples moving horizon estimation with nonlinear model

predictive control. MHE estimates states and parameters by optimising over a recent window of measurements while enforcing the system dynamics and physical bounds [5]. Within that window the estimator identifies two payload-dependent scalars, and from them the controller reconstructs every payload-dependent term in its own model.

1.2 Problem Statement

Consider a quadrotor that follows a three-dimensional figure-eight path, grasps an eccentric payload, transports it and releases it at a selected phase of the trajectory. The controller has no access to the true payload parameters. Only ordinary onboard flight signals are available: odometry, attitude, body rate and rotor-speed telemetry.

Three model errors appear at attachment:

1. the composite mass changes abruptly, shifting the hover thrust and the translational acceleration gain;
2. the payload lever arm increases roll and pitch inertia, reducing effective inner-loop bandwidth;
3. the horizontal centre-of-mass offset creates a non-zero trim moment and consumes part of the available torque authority.

At release all three changes occur in the opposite direction.

The obvious way to handle release is to tell the controller when it happens. The controller in this work in fact *issues* the release command itself, so the information is available for free. The central observation of this dissertation is that using it is a mistake.

A release command is a *request*. The model must describe the *vehicle*. The two diverge whenever a gripper is slow, jams, releases only part of a load, or when a load is lost with no command at all — and a controller that clears its payload model on the command flies a loaded vehicle with an empty model, silently, because nothing in its loop contradicts it.

The central research question is therefore:

Can a quadrotor identify, from its ordinary flight signals alone, that a payload has been grasped and that it has physically left — with no attach or release notification of any kind entering the estimator or the model update — and remain a constrained, well-tracking predictive controller throughout?

This question turns out to have a specific technical obstacle, which Chapter 3 addresses and which shapes the whole design. The natural payload parameterisation — a payload mass and a grasp offset — becomes singular at exactly the moment of interest: as the payload mass goes to zero the offset is undefined, and an estimator holding the offset fixed can only cancel the resulting phantom trim moment by corrupting its mass estimate. The remedy adopted here is to estimate the *first mass moment* $s = m\mathbf{p}\mathbf{r}_{xy}$, the product that the equations of motion actually contain, rather than its two factors.

1.3 Research Objectives

The dissertation has six objectives:

1. derive a consistent 13-state nonlinear quadrotor model with explicit pay-

- load mass, inertia and centre-of-mass entry points, and show which parameterisation of those entry points remains well posed through release;
2. formulate a moving horizon estimator that augments the composite mass and the horizontal first mass moment, deriving the inertia increment and the centre-of-mass offset from that pair without any horizontal attachment geometry;
 3. reconstruct the physical wrench from rotor speeds so that the estimator is never driven by a signal computed from its own output;
 4. formulate a predictive controller whose dynamics, equilibrium-input baseline and inner-loop command scaling are all updated from one atomic, health-gated estimate frame;
 5. define a grasp–release lifecycle in which no attach or release notification is admitted as evidence, and in which an unconfirmed release is held as *unresolved* rather than assumed complete;
 6. evaluate the complete interface in software-in-the-loop against command-triggered and command-armed baselines, under delayed detachment, unsigned loss, partial loss and thrust-map mismatch.

1.4 Scope

The work covers simulation and software-in-the-loop validation of a rigidly attached parcel. Airframe inertia, rotor geometry and the thrust and yaw-reaction coefficients are treated as calibrated plant quantities. The calibrated airframe mass appears in the rigid-body model, in the derived payload mass $m_P = (m_T - m_B)^+$ and in offline scoring.

The information boundary is stated precisely, because a looser statement would be false. *No task-specific payload mass and no attach or release notification enter*

the estimator or the controller's model update. One mass-valued platform constant does remain in the closed loop — the airframe's 0.30 kg payload envelope, used for an attach-time inertia floor and for a physical admissibility bound — and it is disclosed as a specification of the airframe rather than as information about the parcel. Chapter 3 states exactly where it appears.

The scope does not include deformable or sloshing loads, cable-suspended payload dynamics, wind, motor failure, hardware flight, or a formal closed-loop stability proof for the estimator–controller interconnection. These are revisited in Chapter 6.

1.5 Contributions

The contributions are:

1. **A first-moment payload state.** Augmenting $s = m_P r_{xy}$ rather than a mass-and-offset pair removes the release singularity, supplies the centre-of-mass offset as $c_{xy} = s/m_T$ with no grasp geometry, and prevents the estimator from encoding payload *presence* into the mass channel.
2. **A well-posed reduction of the inertia model.** The second-order horizontal terms of the parallel-axis increment are shown to carry a factor $1/m_P$ in the first-moment coordinates, and are discarded for a quantified 2.6 % bias. The remaining mass-dependent coefficient is shown to act as a lever by which the optimiser can dilute a phantom moment, and is frozen within each window to break that derivative.
3. **A non-circular physical-input reconstruction,** including the yaw-reaction channel, so that the estimator observes what the actuators did rather than what the controller intended.

4. **A window vertical-force-balance re-initialisation** that uses only the two velocity endpoints of the window and therefore requires no numerical differentiation, reducing the seed error from 17.1 % to 2.8 % in aggressive flight.
5. **An eventless payload lifecycle** in which the estimator owns presence and may declare release, the controller owns the model and clears it only on persistent confirmation, and an unconfirmed release becomes a hold with a smooth brake to hover rather than an assumption of completion.
6. **A release rule built on a self-formed, frozen moment reference**, with mandatory moment evidence — a requirement introduced after a weaker rule caused real premature releases — and a documented record of what each threshold was selected against.
7. **A single atomic, health-gated payload interface** feeding both the predictive model and a continuous body-rate command scaling with explicit headroom limiting, replacing a design in which parameters arrived on separate topics and the autopilot’s own gains were rewritten in flight.
8. **A paired experimental evaluation** that separates release *semantics* from persistent confirmation, quantifies what eventlessness buys (detection of unsignaled loss) and what it costs (confirmation delay, and a severe sensitivity to thrust-map calibration), and reports the failure modes rather than only the successes.

1.6 Dissertation Organisation

Chapter 2 reviews predictive control, online estimation, inertial-parameter identification and payload-adaptive aerial control. Chapter 3 derives the model and the payload parameter structure, and is where the first-moment argument and its consequences are developed in full. Chapter 4 presents the coupled architecture,

the numerical form of both optimal control problems, the continuous payload interface and the eventless lifecycle. Chapter 5 describes the implementation and reports the frozen experimental campaign, including the conditions under which the interface fails. Chapter 6 summarises the findings and states what remains open.

Chapter 2

Background and Related Work

2.1 Model Predictive Control

MPC repeatedly solves a finite-horizon optimal control problem, applies the first input and shifts the horizon forward. Its practical value comes from combining prediction, multivariable coordination and explicit constraints within one optimisation [1,2]. Linear MPC is mature and computationally efficient, but a single linear model is poorly suited to the large attitude changes and thrust-direction coupling of a quadrotor. NMPC retains the nonlinear dynamics and nonlinear tracking errors directly [3].

The price is computation. Multiple shooting, sequential quadratic programming, Gauss–Newton Hessian approximations and real-time iteration schemes are widely used to make NMPC practical [6]. CasADi provides symbolic differentiation and a high-level optimisation interface [7]; acados generates efficient optimal-control solvers around tailored SQP and QP components [8,9], with HPIPM [10] as the quadratic subproblem solver. The present work uses full SQP with merit-function backtracking, because the real-time iteration variant was observed to diverge silently on this problem.

2.2 Quadrotor NMPC

A quadrotor is differentially flat under common assumptions [11], and many successful controllers exploit position and yaw references with a cascaded inner attitude loop [12]. NMPC can instead optimise collective thrust and body torque against the complete nonlinear rigid-body model, and this formulation becomes advantageous as trajectory curvature and operating envelope increase [4,13].

The controller in this dissertation predicts position, velocity, unit quaternion and body rate. The attitude residual is the vector part of the relative quaternion, which avoids treating the four quaternion components as independent Euclidean coordinates. Input limits represent collective thrust and body-torque authority. This formulation is nominally standard; the distinguishing issue is that the equilibrium input and the plant parameters change at payload attachment and again at release.

2.3 Uncertainty and Adaptation in MPC

Robust MPC protects against bounded uncertainty, while stochastic MPC describes uncertainty probabilistically. Both can preserve margins without identifying the current plant, but conservatism grows when the uncertainty set must cover both an empty and a loaded vehicle. Adaptive MPC instead updates a parameterised model online: certainty-equivalent schemes use the current estimate directly, set-membership schemes maintain a feasible parameter set, and dual-control schemes excite the plant to learn [14,15].

Learning-based MPC extends this using Gaussian processes, neural networks or learned residual models. Data-driven MPC for quadrotors can compensate complex aerodynamic effects [16,17], and neural approximations can reduce online computation [18]; broader surveys of learning for control under safety con-

straints are available [19]. Those methods are valuable when the uncertainty is poorly structured. Payload pickup has stronger prior structure: rigid-body mechanics already specifies how mass and attachment geometry enter the equations. The present work therefore estimates a small physical parameter set rather than learning a general residual map.

A distinct and influential alternative is to decline to identify the payload at all, and to absorb its effect as a lumped matched disturbance. The $\mathcal{L}_1$ -adaptive augmentation of a quadrotor NMPC [20] is the representative of this family, and incremental nonlinear dynamic inversion [21] is its inner-loop counterpart. Such schemes are robust and require no payload model, but their internal quantity is a force or an acceleration, not a mass and a moment. They therefore cannot answer the question this dissertation poses — whether a payload is still attached — because that question is not represented in their state. The comparison is implemented here (an additive lumped translational and rotational disturbance in the prediction model) but is not among the frozen results.

2.4 State and Parameter Estimation

Recursive filters such as the extended Kalman filter propagate a local Gaussian approximation. They are efficient and central to flight-state estimation, but inequality constraints are not naturally hard constraints and abrupt parameter changes violate the local linearisation assumption. MHE estimates a window of past states and disturbances by solving a constrained optimisation; it can impose bounds such as a physically admissible mass interval and can attribute residuals using the same nonlinear model as the controller [1, 5, 22]. Recent work learns the MHE weights themselves [23], which is the principled version of the stage-weight question that Chapter 4 settles negatively for this problem.

A moving window introduces a trade-off that this work encounters directly. A

longer horizon improves noise rejection, but the constraint that makes an augmented parameter identifiable — that it is constant across the window — becomes less defensible as the window spans more of the trajectory. The implemented estimator uses 20 stages at 0.1 s.

2.5 Inertial-Parameter Identification

That the rigid-body equations are *linear* in a particular set of inertial parameters — mass, first mass moment and the six inertia elements — is a classical result in manipulator identification [24]. The first mass moment $s = m p r_{xy}$ used throughout this dissertation is a member of that set, and the argument of Chapter 3 is essentially an application of the classical observation to a case where it has not usually been applied: an aerial vehicle whose payload can *leave*, so that the factored parameterisation becomes singular while the product does not.

Payload-aware estimation for multirotors has previously used force–torque sensing, external observers or explicit mass estimators [25–28], and slung-load and aerial-manipulation work treats the payload as a dynamic subsystem of its own [29–31]. Both families assume that whether a payload is present is known. That assumption is what this dissertation removes.

2.6 Change Detection and Event-Triggered Estimation

Deciding that a plant has changed, from a residual, is the subject of classical change-detection theory [32], and event-triggered estimation is an established response to it [33]. The predecessor of the present system used exactly this structure: a persistent step in the reconstructed collective thrust triggered a de-weighting of the pre-event stages of the window.

Chapter 4 reports why that structure was retired for this problem. A collective-thrust residual cannot distinguish a payload change from a trajectory-mode change, a sustained vertical gust, or an aggressive manoeuvre, because all of them move the same scalar. Any decision resting on that signal alone inherits the ambiguity. The deployed design instead requires evidence from the *rotational* channel — a collapse of the first moment relative to a self-formed reference — which is the one channel that measured cleanly in the development data. A formal comparison against a CUSUM detector on the same signals, under a matched false-alarm constraint, is not performed and is listed among the gaps.

2.7 Research Gap

The literature contains strong results on quadrotor NMPC, on learned residual dynamics, on inertial-parameter identification and on payload-aware aerial control. Three things are missing at their intersection, and this dissertation addresses them.

First, **a payload parameterisation that survives release.** Existing payload-adaptive formulations assume attachment and estimate parameters conditioned on it. None of them is well posed in the limit $m_p \rightarrow 0$ with a non-zero assumed lever arm, which is the regime in which every release occurs.

Second, **a model-update rule that does not consult the event.** Systems that grasp and release overwhelmingly treat the gripper command, or a contact sensor, as ground truth for the model. The mismatch between command and physical detachment is not usually measured, and the failure it produces — a loaded vehicle flown with an empty model — is silent.

Third, **an honest account of what such a design costs.** Refusing the command

delays confirmation, and the delay is bimodal rather than merely larger. Estimating an absolute mass from an absolute thrust makes the whole interface sensitive to a calibration constant. Both are quantified in Chapter 5 rather than asserted to be small.

The dissertation consequently treats estimation and control as one architecture rather than two algorithms. Estimation is judged by whether its outputs are identifiable and useful for control; control is judged by whether it remains consistent with the quantities being estimated, and by whether it knows when it does not know.

Chapter 3

Modeling and Parameter Structure

A quadrotor that picks up a parcel does not merely become heavier. It also becomes harder to rotate, and its centre of mass no longer lies on the thrust axis. These are three distinct changes to the equations of motion, and a controller that compensates only the first will fail in ways that mass adaptation cannot explain.

This chapter derives the model that the estimator and the controller share, and then makes explicit which of its parameters are identified online and which are not. The structural claim of the chapter — and the reason the method needs so little sensing — is that the three payload-induced quantities are not three independent unknowns, but neither are they functions of the mass alone. Two scalars suffice: the composite mass m_T and the horizontal *first mass moment* $s = m_P r_{xy}$. From this pair the centre-of-mass offset, the trim moment and the dominant roll/pitch inertia increment all follow by rigid-body algebra, without the estimator ever being told the grasp offset, and without any quantity becoming undefined when the payload leaves.

Section 3.1 fixes frames and conventions, Sections 3.2–3.3 derive the kinematics and dynamics, and Section 3.4 derives the classical payload parameterisation in terms of mass and a known lever arm. Section 3.5 then shows why that parameterisation is the wrong coordinate system for a vehicle that must also *release* its load, and replaces it with the first-moment parameterisation that the

deployed estimator uses. Sections 3.6–3.8 describe how the parameters are measured and consumed, and Sections 3.9–3.10 defend the design and declare its approximations.

3.1 Frames, Notation, and Conventions

Two reference frames are used throughout. The world frame $\mathcal{W}$ is East–North–Up (ENU), so that gravity acts along $-z$. The body frame $\mathcal{B}$ is Front–Left–Up (FLU) with its origin at the geometric centre of the rotor disc — *not* at the centre of mass. This choice is deliberate and consequential: the thrust vector passes through the body origin by construction, so any offset between that origin and the true centre of mass appears explicitly as a moment (Section 3.3) rather than being absorbed silently into the frame definition.

The state vector and the control input are

$$x = [\mathbf{p}; \mathbf{v}; \mathbf{q}; \boldsymbol{\omega}] \in \mathbb{R}^{13}, \quad u = [T; \boldsymbol{\tau}] \in \mathbb{R}^4, \quad (3.1)$$

where $\mathbf{p}$ and $\mathbf{v}$ are the position and velocity in $\mathcal{W}$, $\mathbf{q}$ is the unit quaternion encoding the attitude of $\mathcal{B}$ relative to $\mathcal{W}$ (Section 3.1.1), $\boldsymbol{\omega}$ is the body rate in $\mathcal{B}$, T is the collective thrust along the body z axis, and $\boldsymbol{\tau}$ is the body torque. We abbreviate $\mathbf{e}_3 = [0, 0, 1]^\top$, and write m_B , m_P and $m_T = m_B + m_P$ for the bare-airframe, payload and composite masses. A subscript xy denotes the horizontal part of a body-frame vector, so that $\mathbf{r}_{xy}$ and $\mathbf{c}_{xy}$ are the horizontal parts of $\mathbf{r}_p$ and $\mathbf{c}$ respectively.

3.1.1 Attitude representation

Attitude is represented by a unit quaternion under the *Hamilton* convention with the scalar part first,

$$\mathbf{q} = [q_w, q_x, q_y, q_z]^\top, \quad \|\mathbf{q}\| = 1, \quad (3.2)$$

encoding the rotation from $\mathcal{B}$ to $\mathcal{W}$. The corresponding rotation matrix is

$$R(\mathbf{q}) = \begin{bmatrix} q_w^2 + q_x^2 - q_y^2 - q_z^2 & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & q_w^2 - q_x^2 + q_y^2 - q_z^2 & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & q_w^2 - q_x^2 - q_y^2 + q_z^2 \end{bmatrix}. \quad (3.3)$$

Three conventions are fixed here and used throughout: the Hamilton product, the scalar-first component order of (3.2), and $\boldsymbol{\omega}$ resolved in $\mathcal{B}$. The last of these determines the side on which the body rate enters (3.6).

One scalar derived from (3.3) is used repeatedly: the cosine of the tilt angle between the body z axis and the vertical,

$$\cos \theta = \mathbf{e}_3^\top R(\mathbf{q}) \mathbf{e}_3 = 1 - 2(q_x^2 + q_y^2), \quad (3.4)$$

which is the fraction of the collective thrust that acts vertically.

3.2 Rigid-Body Kinematics

Translational kinematics is the definition of velocity,

$$\dot{\mathbf{p}} = \mathbf{v}. \quad (3.5)$$

Attitude kinematics follows from the quaternion product of the attitude with the pure quaternion formed from the body rate. Writing $\mathbf{q} = (q_w, \mathbf{q}_v)$ with $\mathbf{q}_v = [q_x, q_y, q_z]^\top$, and using the Hamilton product $(a_w, \mathbf{a}_v) \otimes (b_w, \mathbf{b}_v) = (a_w b_w - \mathbf{a}_v \cdot \mathbf{b}_v, a_w \mathbf{b}_v + b_w \mathbf{a}_v + \mathbf{a}_v \times \mathbf{b}_v)$, we obtain

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes [0, \boldsymbol{\omega}] = \frac{1}{2} \begin{bmatrix} -\mathbf{q}_v \cdot \boldsymbol{\omega} \\ q_w \boldsymbol{\omega} + \mathbf{q}_v \times \boldsymbol{\omega} \end{bmatrix} = \frac{1}{2} \boldsymbol{\Xi}(\mathbf{q}) \boldsymbol{\omega}, \quad (3.6)$$

where the second equality expands the product and the third collects the result into the 4×3 matrix

$$\boldsymbol{\Xi}(\mathbf{q}) = \begin{bmatrix} -q_x & -q_y & -q_z \\ q_w & -q_z & q_y \\ q_z & q_w & -q_x \\ -q_y & q_x & q_w \end{bmatrix}. \quad (3.7)$$

The implementation uses (3.7), a plain matrix–vector product that is linear in $\boldsymbol{\omega}$. Note that the body rate enters on the *right* of the product in (3.6). Had $\boldsymbol{\omega}$ been resolved in $\mathcal{W}$, the correct expression would be $\dot{\mathbf{q}} = \frac{1}{2} [0, \boldsymbol{\omega}] \otimes \mathbf{q}$ instead, differing by a conjugation by $\mathbf{q}$.

The matrix form also exposes a property that the product form hides. For any unit quaternion,

$$\boldsymbol{\Xi}(\mathbf{q})^\top \boldsymbol{\Xi}(\mathbf{q}) = I_3, \quad \mathbf{q}^\top \boldsymbol{\Xi}(\mathbf{q}) = \mathbf{0}^\top, \quad (3.8)$$

as may be checked directly from (3.7) using $\|\mathbf{q}\| = 1$. The second identity gives $\mathbf{q}^\top \dot{\mathbf{q}} = 0$, and therefore

$$\frac{d}{dt} \|\mathbf{q}\|^2 = 2 \mathbf{q}^\top \dot{\mathbf{q}} = 0 : \quad (3.9)$$

the unit-norm constraint is preserved by the kinematics themselves, rather than maintained by repeated renormalisation. This matters inside the estimator, which integrates (3.6) across the whole window before any measurement is compared: a systematic drift of $\|\mathbf{q}\|$ would appear as an attitude residual and be attributed to the payload parameters.

Neither (3.5) nor (3.6) contains a mass, an inertia, or any other physical parameter: kinematics is pure geometry. *The payload therefore enters the model only through the dynamics*, which confines the entire parameter discussion of this chapter to the three equations of Section 3.3.

3.3 Rigid-Body Dynamics

3.3.1 Translational dynamics

Newton's second law in $\mathcal{W}$, with thrust rotated from the body frame and a linear aerodynamic drag term, gives

$$\dot{\mathbf{v}} = \frac{1}{m_T} \left(R(\mathbf{q}) \mathbf{e}_3 T - k_d \mathbf{v} \right) - g \mathbf{e}_3, \quad (3.10)$$

where m_T is the *composite* mass of airframe plus payload, k_d is a fixed drag coefficient, and g is the gravitational acceleration.

Equation (3.10) is where the mass — and *only* the mass — enters, and it enters in exactly one role: as the scalar gain $1/m_T$ from thrust to acceleration. This single fact underwrites the observability argument of the whole method. If a thrust value can be obtained independently of the model, then the measured acceleration determines m_T uniquely by (3.10), and it does so in ordinary near-hover flight, with no dedicated excitation manoeuvre. The qualification “independently of the model” is essential and is the subject of Section 3.6.

It is worth recording the special case of (3.10) along the vertical, because Section 3.7.3 uses it directly. Projecting (3.10) onto $\mathbf{e}_3$ and neglecting the vertical component of the drag term gives the vertical force balance

$$m_T(g + a_z) = T \cos \theta, \quad (3.11)$$

with $\cos \theta$ from (3.4) and $a_z = \dot{v}_z$.

3.3.2 Rotational dynamics

Let $J_B = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ denote the inertia of the bare airframe about its own centre of mass. We assume throughout that the airframe is balanced about the body origin of Section 3.1, so that the two points coincide when no payload is attached. This is an assumption about the vehicle, not a consequence of the frame choice, and it is load-bearing: Section 3.5 relies on it to attribute the entire first mass moment to the payload.

Let J_T denote the composite inertia, derived in Sections 3.4.2 and 3.5. Euler's equation for a rigid body, written about the composite centre of mass and in the body frame, is

$$\dot{\omega} = J_T^{-1} \left(\tau + \tau_{\text{com}}(c_{xy}) - \omega \times J_T \omega \right). \quad (3.12)$$

The term $\omega \times J_T \omega$ is the usual gyroscopic coupling. The term τ_{com} is derived next.

That (3.12) is written *about the composite centre of mass* is not a stylistic choice. Euler's equation takes this form only when the inertia tensor and the applied moments refer to the same point, and that point is either the centre of mass or a point fixed in the inertial frame; a free-flying vehicle has no such fixed point, so the centre of mass is the only admissible choice. Both J_T and τ_{com} must therefore be computed about the composite CoM — which *moves* relative to the body frame when a payload is attached, and that displacement is precisely why τ_{com} exists. Mixing the two reference points is a common and silent modelling error.

3.3.3 The thrust eccentricity moment

Thrust acts at the rotor-disc centre, which is the body origin. When a payload is attached the composite centre of mass moves to some $\mathbf{c} = [c_x, c_y, c_z]^\top \neq \mathbf{0}$, and the thrust therefore no longer passes through it. The moment about the CoM is the cross product of the position of the application point *relative to the CoM*, namely $-\mathbf{c}$, with the force $\mathbf{e}_3 T$:

$$\begin{aligned} \tau_{\text{com}}(\mathbf{c}_{xy}) &= (-\mathbf{c}) \times \mathbf{e}_3 T = - \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix} \times \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} \\ &= - \begin{bmatrix} c_y T - 0 \\ 0 - c_x T \\ 0 - 0 \end{bmatrix} = \begin{bmatrix} -c_y T \\ +c_x T \\ 0 \end{bmatrix}. \end{aligned} \quad (3.13)$$

Two properties of (3.13) are used repeatedly in what follows.

The vertical offset c_z does not appear. Its contribution to the cross product vanishes identically, since a vertical displacement of the centre of mass is collinear with the body- z thrust; gravity acts at the centre of mass and likewise exerts no moment about it. The offset relevant to the attitude dynamics is therefore two-dimensional by construction rather than by truncation.

The moment τ_{com} is non-zero in level hover and does not average to zero along a trajectory. For the payload of Section 3.4.3 it amounts to roughly two thirds of the available roll authority, so station-keeping with an eccentric load requires a persistent trim torque. A prediction model that omits (3.13) is therefore in error by a constant moment rather than by a small one, and plans as though no trim torque were required. Section 3.8 shows that the same term must additionally appear in the input reference of the cost function.

3.3.4 Structure of the parameter entry points

Collecting (3.10), (3.12) and (3.13), the model has the property on which the rest of this dissertation is built:

Each payload-dependent parameter has exactly one point of entry. The composite mass appears as $1/m_T$ in the translational dynamics; the inertia increment appears only inside J_T in the rotational dynamics; the centre-of-mass offset appears only in the eccentricity moment, which is also a rotational quantity.

This separation is what makes the parameters individually attributable to distinct physical channels — the mass to the thrust–acceleration relation, the inertia to angular acceleration, the offset to the steady-state torque balance. It is exploited directly in Sections 3.7 and 3.9.

3.4 Payload-Induced Parameters

We now derive c_{xy} and the inertia increment. The payload is modelled as a point mass m_P rigidly attached at a body-frame offset $\mathbf{r}_p = [r_x, r_y, r_z]^\top$ with $r_z < 0$ (the load hangs below the airframe). Define the composite mass and the *reduced mass*

$$m_T = m_B + m_P, \quad \mu = \frac{m_B m_P}{m_B + m_P} = \frac{m_B m_P}{m_T}. \quad (3.14)$$

This section treats $(m_P, \mathbf{r}_p)$ as the parameterisation, because it is the physically transparent one. Section 3.5 then shows that this pair is not what an estimator can or should carry, and re-expresses every result below in coordinates that survive release.

3.4.1 Centre-of-mass offset

The composite centre of mass is the mass-weighted mean of the two constituent positions. Taking the airframe centre of mass at the body origin and the payload at $\mathbf{r}_p$,

$$\mathbf{c} = \frac{m_B \cdot \mathbf{0} + m_P \cdot \mathbf{r}_p}{m_T} = \frac{m_P}{m_T} \mathbf{r}_p, \quad (3.15)$$

whose horizontal components are the pair required by (3.13):

$$\mathbf{c}_{xy} = [c_x, c_y]^\top = \frac{m_P}{m_T} [r_x, r_y]^\top. \quad (3.16)$$

3.4.2 Inertia increment and the reduced mass

The inertia increment requires more care than the parallel-axis theorem applied naively, because *the centre of mass itself moves when the payload is attached*. The theorem must be applied about the *new* centre of mass (3.15), and both bodies contribute, since the airframe no longer sits at the CoM either.

Consider a single axis and let d denote the corresponding lever arm. From (3.15), the distances of the two bodies from the composite centre of mass are

$$\underbrace{\|\mathbf{0} - \mathbf{c}\|}_{\text{airframe}} = \frac{m_P}{m_T} d, \quad \underbrace{\|\mathbf{r}_p - \mathbf{c}\|}_{\text{payload}} = \left(1 - \frac{m_P}{m_T}\right) d = \frac{m_B}{m_T} d. \quad (3.17)$$

Note that the payload's distance is *smaller* than d . Adding the two parallel-axis contributions and simplifying,

$$\begin{aligned} \Delta J^{(d)} &= m_B \left(\frac{m_P}{m_T} d\right)^2 + m_P \left(\frac{m_B}{m_T} d\right)^2 \\ &= \frac{d^2}{m_T^2} (m_B m_P^2 + m_P m_B^2) = \frac{d^2}{m_T^2} m_B m_P (m_P + m_B) \\ &= \frac{m_B m_P}{m_T} d^2 = \mu d^2. \end{aligned} \quad (3.18)$$

The factor $(m_B + m_P)$ produced by the sum cancels one power of m_T in the denominator, and the two contributions collapse onto the reduced mass (3.14) — the same quantity that governs the two-body problem in classical mechanics, and for the same reason: the inertia of a rigid pair depends only on their *relative* configuration.

Equation (3.18) also identifies a natural error. Writing $\Delta J^{(d)} = m_P d^2$ — treating the payload as orbiting a fixed body origin, and forgetting both the airframe's own contribution and the displacement of the CoM — overestimates the increment by the factor m_T/m_B , which is 14.5 % at the payload considered below. Two limiting cases confirm (3.18): as $m_P \ll m_B$ we recover $\mu \rightarrow m_P$ (a light load barely moves the CoM, and the naive expression becomes correct), while as $m_P \gg m_B$ we obtain $\mu \rightarrow m_B$ (a heavy load anchors the CoM and it is the airframe that revolves about it).

The same computation carried out for the full tensor rather than one axis gives the increment in closed form. Writing I_3 for the identity,

$$\begin{aligned} \Delta J(m_P, \mathbf{r}_P) &= m_B \left(\|\mathbf{c}\|^2 I_3 - \mathbf{c} \mathbf{c}^\top \right) + m_P \left(\|\mathbf{r}_P - \mathbf{c}\|^2 I_3 - (\mathbf{r}_P - \mathbf{c})(\mathbf{r}_P - \mathbf{c})^\top \right) \\ &= \mu \left(\|\mathbf{r}_P\|^2 I_3 - \mathbf{r}_P \mathbf{r}_P^\top \right), \end{aligned} \quad (3.19)$$

by exactly the cancellation of (3.18) applied entrywise. Written out,

$$\Delta J = \mu \begin{bmatrix} r_y^2 + r_z^2 & -r_x r_y & -r_x r_z \\ -r_x r_y & r_x^2 + r_z^2 & -r_y r_z \\ -r_x r_z & -r_y r_z & r_x^2 + r_y^2 \end{bmatrix}. \quad (3.20)$$

Each diagonal entry is (3.18) evaluated with the lever arm of that axis, and each off-diagonal entry is the product $-\mu r_i r_j$ of the two arms it couples. Reading the diagonal off gives the familiar per-axis increments

$$\Delta J_{xx} = \mu (r_y^2 + r_z^2), \quad \Delta J_{yy} = \mu (r_x^2 + r_z^2), \quad \Delta J_{zz} = \mu (r_x^2 + r_y^2). \quad (3.21)$$

The composite inertia is $J_T = J_B + \Delta J$. Section 3.5.3 regroups the same six entries into the coordinates that survive release.

3.4.3 Magnitudes on the experimental platform

To fix ideas, consider a 0.3 kg payload attached at $r_p = [0, 0.109, -0.47]$ m on the platform of this work ($m_B = 2.0643$ kg, $J_{xx} = 0.0142$ kg m², roll/pitch torque authority 0.5 N m). Equations (3.14), (3.16) and (3.21) give

$$\begin{aligned} \mu &= 0.262 \text{ kg}, & \Delta J_{xx} &= 0.0610 \text{ kg m}^2, \\ \frac{J_{xx} + \Delta J_{xx}}{J_{xx}} &= 5.30, & c_y &= 13.8 \text{ mm}, \end{aligned} \quad (3.22)$$

and hence a constant hover trim torque

$$|c_y| m_T g = 0.32 \text{ N m} \approx 64\% \text{ of the roll authority.} \quad (3.23)$$

These numbers state the problem this dissertation addresses more sharply than any qualitative argument. A 0.3 kg parcel on a 2.06 kg airframe is a 15 % change of mass — but simultaneously a more than *five-fold* change of roll inertia and a standing demand on two thirds of the torque margin. Adapting the mass alone accounts for neither of the latter two. Note also that the r_z^2 term contributes 97.4 % of ΔJ_{xx} : the increment is dominated by how far *below* the airframe the load hangs, not by how far it is off-centre. That asymmetry is exploited repeatedly below, because r_z turns out to be the one geometric quantity the estimator cannot observe and, fortunately, the one whose error matters least.

3.5 The First Mass Moment as the Payload State

Everything in Section 3.4 is expressed in the coordinates $(m_P, \mathbf{r}_P)$. Those are the natural coordinates for *describing* a grasp. They are the wrong coordinates for *estimating* one, and the reason is visible in (3.16): the map from $(m_P, \mathbf{r}_P)$ to the observable quantities is not invertible in the limit that matters most.

3.5.1 Why the offset is the wrong coordinate

Consider what each quantity does as the payload is released, $m_P \rightarrow 0$.

- The moment τ_{com} of (3.13) tends to zero, because $\mathbf{c}_{xy} = (m_P/m_T)\mathbf{r}_{xy} \rightarrow \mathbf{0}$. This is physical and benign.
- The horizontal offset $\mathbf{r}_{xy}$ itself becomes *undefined*. There is no fact of the matter about where a payload of zero mass is attached, and no measurement can supply one: every observable depends on $\mathbf{r}_{xy}$ only through the product $m_P\mathbf{r}_{xy}$.

An estimator parameterised by $(m_P, \mathbf{r}_{xy})$ therefore carries a direction that becomes unidentifiable exactly at release. If $\mathbf{r}_{xy}$ is instead held fixed as a prior, as in a conventional coupled model, the situation is worse rather than better: the lever arm stays in the model after the load has gone, so the model predicts a phantom trim moment $(m_P/m_T)\mathbf{r}_{xy}T$ that the measurements contradict. The only remaining way for the optimiser to cancel that phantom is to drive m_P — and therefore m_T — toward or below the bare-airframe value. This is not hypothetical: in the mass-only formulation that preceded the present design, 14 of 55 post-release samples pinned at the mass lower bound, and the recovered composite mass was biased low by 2.81 %. The information “is there a payload at all” had been encoded into the mass channel, where it corrupted the quantity the controller most needs.

The remedy is a change of coordinates. Define the horizontal *first mass moment* of the composite body about the body-frame origin,

$$\mathbf{s} = \begin{bmatrix} s_x \\ s_y \end{bmatrix} = m_P \mathbf{r}_{xy} \quad [\text{kg m}]. \quad (3.24)$$

Because the bare airframe's centre of mass coincides with the body origin, the airframe contributes nothing to the first moment and the entire moment is the payload's, whatever grasp offset the gripper happens to realise. Equation (3.24) is the standard linear parameterisation of rigid-body inertial parameters [24]: the equations of motion contain the *product* $m_P \mathbf{r}_{xy}$ and never the two factors separately, which is precisely why the product, and not the offset, is the quantity to estimate.

3.5.2 The two channels the first moment drives

Substituting (3.24) into (3.16) gives the first payoff:

$$\mathbf{c}_{xy} = \frac{m_P}{m_T} \mathbf{r}_{xy} = \frac{1}{m_T} (m_P \mathbf{r}_{xy}) = \frac{\mathbf{s}}{m_T}. \quad (3.25)$$

The centre-of-mass offset is available from the two estimated states alone. No grasp geometry is required, and — critically — m_P does not appear in a denominator, so (3.25) stays well defined and continuous as the payload leaves. Combining (3.25) with (3.13), the trim moment consumed by the model is

$$\boldsymbol{\tau}_{\text{com}} = \frac{T}{m_T} \begin{bmatrix} -s_y \\ +s_x \\ 0 \end{bmatrix}. \quad (3.26)$$

The second channel is the off-diagonal inertia. Taking the (1,3) entry of (3.19)

and using (3.14) and (3.24),

$$J_{xz} = -\mu r_x r_z = -\frac{m_B m_P}{m_T} r_x r_z = -\frac{m_B}{m_T} (m_P r_x) r_z = -\frac{m_B}{m_T} s_x r_z, \quad (3.27)$$

and identically $J_{yz} = -(m_B/m_T) s_y r_z$. The payload mass has cancelled exactly: these terms need only s and the vertical arm r_z .

3.5.3 Re-expressing the inertia increment

It is convenient to split (3.19) into the three groups that behave differently under the change of coordinates. Define

$$A = \mu r_z^2, \quad B_i = \mu r_i r_z = \frac{m_B}{m_T} s_i r_z, \quad C_{ij} = \mu r_i r_j \quad (i, j \in \{x, y\}), \quad (3.28)$$

so that the composite inertia of (3.19) reads

$$J_T = J_B + (A + C_{xx} + C_{yy}) I_3 - \begin{bmatrix} C_{xx} & C_{xy} & B_x \\ C_{xy} & C_{yy} & B_y \\ B_x & B_y & A \end{bmatrix}. \quad (3.29)$$

Expanding the (1,1) entry recovers $J_{xx} + \mu(r_y^2 + r_z^2)$ and the (3,3) entry recovers $J_{zz} + \mu(r_x^2 + r_y^2)$, confirming (3.21).

In the new coordinates the three groups have sharply different status.

1. B_i requires *only* s and r_z , by (3.27).
2. $A = (m_B m_P / m_T) r_z^2$ requires $m_P = m_T - m_B$ and r_z , both available: the composite mass is estimated and r_z is a weak prior. No horizontal geometry is needed.

3. C_{ij} is the problem. Writing it in the new coordinates,

$$C_{ij} = \mu r_i r_j = \frac{m_B m_P}{m_T} \cdot \frac{s_i}{m_P} \cdot \frac{s_j}{m_P} = \frac{m_B}{m_T m_P} s_i s_j. \quad (3.30)$$

Equation (3.30) is the formal statement of the difficulty. As $m_P \rightarrow 0$ with s not yet driven to zero by the data — exactly the state of affairs during the first fraction of a second after release — the coefficient $m_B/(m_T m_P)$ diverges. The model then admits a region in which a vanishingly light payload hangs on an arbitrarily long lever arm: physically absurd, numerically attractive to an optimiser looking for a way to explain a residual, and ill-conditioned in exactly the direction the estimator is trying to resolve.

The deployed model therefore *discards* the C_{ij} group, setting $C_{xx} = C_{yy} = C_{xy} = 0$ in (3.29). What remains,

$$J_T = J_B + A I_3 - \begin{bmatrix} 0 & 0 & B_x \\ 0 & 0 & B_y \\ B_x & B_y & A \end{bmatrix}, \quad \Delta J_{xx} = \Delta J_{yy} = A = \frac{m_B m_P}{m_T} r_z^2, \quad (3.31)$$

contains no term that requires m_P and r_{xy} to be known separately. The price is quantified exactly: the roll increment loses $C_{yy} = \mu r_y^2$ out of $\mu(r_y^2 + r_z^2)$, a relative error

$$\frac{C_{yy}}{A + C_{yy}} = \frac{r_y^2}{r_z^2 + r_y^2}, \quad (3.32)$$

which at the platform geometry of (3.22) ($r_y = 0.109\text{m}$, $r_z = -0.47\text{m}$) is 2.6 %. That is the cost of removing a degenerate direction from the model, and it buys a formulation in which the post-release behaviour is well posed. Offline replay of a synthetic grasp–release sequence makes the trade concrete: retaining C_{ij} produced a composite-mass estimate that climbed monotonically to 3.17 kg after release and did not return, with s_y changing sign frame to frame, whereas the reduced model of (3.31) settled at -0.34% of the bare airframe with s and ΔJ decaying together and no bound contact.

3.5.4 The inertia lever and the frozen coefficient

Removing C_{ij} eliminates one pathological coupling between mass and geometry, but not the only one. The remaining A term is still linear in m_P , and that is enough to give the optimiser a second lever on the mass channel.

The mechanism is as follows. Suppose a phantom horizontal offset c_y survives briefly after release. It generates a predicted roll acceleration of magnitude

$$|\dot{\omega}_x^{\text{ghost}}| \approx \frac{|c_y|T}{J_{xx} + A(m_P)}, \quad (3.33)$$

which the measurements contradict, since the real vehicle is not rolling. The numerator is bounded, because $\|c_{xy}\| \leq \|r_{xy}\|$ is a geometric limit. The denominator, however, grows linearly in m_P , and grows fast: $r_z^2 = 0.22 \text{ m}^2$ is some fifteen times $J_{xx} = 0.0142 \text{ kg m}^2$. *Increasing the mass estimate therefore dilutes the phantom moment.* The optimiser will do so if the rotational residual it thereby removes outweighs the translational residual it thereby creates — and in offline replay of a 0.3 kg release it did exactly that, driving $\hat{m}$ to 3.17 kg (+53%) and accepting a -3.4 ms^{-2} translational inconsistency to suppress the angular residual. The same disease has a second outlet: with the lower bound active it instead pushes the estimate down onto $m_{\min}$. Either way, the mass has been recruited as a geometry knob.

The implementation therefore evaluates A from the previous window's converged mass estimate $\hat{m}^-$, supplied to the current solve as a parameter rather than computed from the decision variable:

$$A = \frac{m_B (\hat{m}^- - m_B)^+}{\hat{m}^-} r_z^2, \quad \left. \frac{\partial A}{\partial m_T} \right|_{\text{window}} = 0. \quad (3.34)$$

Within a single solve A is then a constant and the gradient path of the preceding paragraph is absent; between solves it is refreshed at the 10 Hz window rate, so its value continues to follow the true mass with one window of lag.

Table 3.1 lists the four treatments of A that the implementation supports and that Chapter 5 compares. They differ in two independent respects: whether the derivative is broken, and whether the retained value is correct. Only the lagged form satisfies both, and it is the deployed default.

Table 3.1: Treatments of the inertia coefficient A . Breaking the derivative removes the gradient path of (3.33); retaining a correct value is a separate requirement, which the last two forms fail.

Treatment	A evaluated from	$\partial A / \partial m_T$	Retained value
live	the decision variable m_T	non-zero	exact
lagged (deployed)	previous solution $\hat{m}^-$	zero	exact to one window
envelope	payload envelope m_P^{env}	zero	biased by m_P^{env} / m_P
omitted	— ($A \equiv 0$)	zero	zero; diagnostic only

The corresponding implementation settings are listed in Appendix A.

3.5.5 Smooth positive part

Two of the expressions above use $m_P = (m_T - m_B)^+$, and the mass decision variable is permitted to fall slightly below m_B to leave room for calibration error. A hard $\max(\cdot, 0)$ is not differentiable at the origin, which makes a Gauss–Newton step chatter in precisely the regime — m_P near zero — where release decisions are taken. The implementation therefore uses the smooth positive part

$$m_P^+ = \frac{1}{2} \left(\Delta + \sqrt{\Delta^2 + \varepsilon^2} \right), \quad \Delta = m_T - m_B, \quad \varepsilon = 0.02 \text{ kg}, \quad (3.35)$$

which satisfies $m_P^+ \geq \frac{\varepsilon}{2} > 0$, is smooth, and agrees with $\max(\Delta, 0)$ to within $\varepsilon/2$ everywhere. Positivity matters beyond smoothness: a negative m_P would make $\mu < 0$ in (3.28) and could render J_T indefinite, at which point J_T^{-1} in (3.12) diverges.

3.6 Model-Independent Reconstruction of the Physical Input

The thrust and torque appearing in (3.10) and (3.12) must be known to the estimator. They are reconstructed from measured rotor speeds ω_i , the known rotor positions (x_i, y_i) and the spin directions $d_i \in \{\pm 1\}$, via the quadratic rotor model $F_i = k_f \omega_i^2$:

$$T_{\text{phys}} = \sum_i F_i, \quad \tau_{\text{phys}} = \begin{bmatrix} \sum_i y_i F_i \\ -\sum_i x_i F_i \\ -k_m \sum_i d_i F_i \end{bmatrix}, \quad (3.36)$$

with $|x_i| = |y_i| = 0.174\text{m}$, $k_f = 8.55 \times 10^{-6} \text{Ns}^2$ and yaw-reaction coefficient $k_m = 0.016\text{m}$. The first two moment components are lever-arm products; the third is the aerodynamic reaction torque, proportional to rotor thrust and opposed to the rotor's own spin.

This is a wrench, not a state estimate. Equation (3.36) returns the wrench the *actuators generate*. It deliberately excludes $J_T \dot{\omega}$ and rotor gyroscopic terms, which is exactly what makes it usable as the known input u^{phys} of the estimator rather than as a second, competing estimate of the state. Rotor speeds are sampled at 250Hz and averaged over each 10Hz estimation interval, which is the zero-order-hold-consistent discretisation of (3.10) and (3.12).

Why the commanded input cannot be used. Let the controller compute its thrust command from an assumed mass $\tilde{m}$, so that in hover $T_{\text{cmd}} = \tilde{m}g$. Feeding T_{cmd} into (3.10) and solving for the mass yields $m_T = T_{\text{cmd}}/g = \tilde{m}$ — for *any* $\tilde{m}$ whatsoever. The relation is satisfied identically, the residual carries no information, and the mass is unobservable: the estimate would simply reproduce whatever the controller already believed. Any inversion error, saturation or

motor lag between command and rotor is likewise invisible. Equation (3.36) breaks this circularity because rotor speeds are a physical measurement into which no model state enters, taken downstream of allocation and the motor dynamics. This is a recurring principle in what follows: an adaptive loop must be fed by a signal that is not itself a product of the adaptation.

The yaw channel makes the point quantitatively. Across twenty batches of recorded flights, the roll and pitch torques actually executed track the controller's intended values closely (median correlation +0.93 and +0.70, slopes 1.08 and 1.10), but the executed yaw torque is only 0.32 of the intended value: a quadrotor's yaw authority comes solely from reaction torque and is an order of magnitude weaker than its roll/pitch authority. An estimator fed the intended yaw torque is fed a moment three times larger than the one the vehicle actually experienced. Reconstructing all four components from (3.36) removes the last place where the estimator could learn what the controller was thinking.

Thrust-to-throttle inversion and the actuator envelope. The same rotor model defines what the vehicle can actually produce, and therefore the input constraint set of the controller. The flight stack accepts a normalised throttle η , which the electronic speed controllers map affinely to rotor speed before the quadratic thrust law applies:

$$\omega(\eta) = \omega_{\min} + \Delta\omega \eta, \quad T(\eta) = K \omega(\eta)^2, \quad K = 4k_f, \quad (3.37)$$

with $\omega_{\min} = 150\text{rad s}^{-1}$ and $\Delta\omega = 850\text{rad s}^{-1}$. Inverting (3.37) gives the command actually published,

$$\eta(T) = \frac{\sqrt{T/K} - \omega_{\min}}{\Delta\omega}, \quad (3.38)$$

which is exact at every mass, unlike the linear hover-referenced heuristic it replaced. Because η is clipped to $[0.05, 0.95]$ — the upper limit reserving 5%

of the command range so that the rate loop retains differential authority when all four rotors are near saturation — the physically attainable collective thrust is

$$T \in [T(0.05), T(0.95)] = [1.27\text{N}, 31.35\text{N}]. \quad (3.39)$$

Section 3.8 uses (3.39) as the thrust box of the optimal control problem. This matters more than it may appear: the earlier implementation used $[0.5\text{N}, 2m_{BG} = 40.5\text{N}]$, a box derived from the vehicle's *weight* rather than its actuators, and therefore over-stated the available thrust by 29%. Planning against authority the vehicle does not have is not a conservative error; the surplus is removed silently by the clip in (3.38), so the planned recovery acceleration is never delivered, the tracking error grows, and the next solve asks for still more thrust.

3.7 Estimation: Mass and First Moment as the Online Degrees of Freedom

3.7.1 Augmented state and estimator formulation

The moving-horizon estimator augments both payload states into the state vector,

$$x_{\text{aug}} = [x; m_T; s_x; s_y] \in \mathbb{R}^{16}, \quad \dot{m}_T = 0, \quad \dot{s} = 0, \quad (3.40)$$

holding both constant across the horizon so that a window of measurements constrains a single value of each rather than a free trajectory. Process noise $w \in \mathbb{R}^{13}$ is admitted on the physical states only; adding noise to the payload channels would allow them to drift within the window and would dissolve precisely the constraint that makes the estimate meaningful. Chapter 4 gives the numerical form of the optimisation, its weights and its bounds; the present section states only what the states mean and which residual identifies which.

The geometry does *not* appear in the decision vector. The estimator's runtime parameter vector carries the reconstructed wrench, the previous window's mass estimate required by (3.34), and the weak vertical arm:

$$p_{\text{MHE}} = [T_{\text{phys}}; \tau_{\text{phys}}; \hat{m}^-; \cdot; r_z] \in \mathbb{R}^7. \quad (3.41)$$

Inside the model, c_{xy} and J_T are evaluated from the decision variables by (3.25) and (3.31), so both the translational and the rotational Jacobians of the payload states are consistent within a single solve. This is a structural point, not a numerical one: an implementation that computed c_{xy} and ΔJ *outside* the optimiser from the previous mass estimate would close an outer fixed-point iteration,

$$\hat{m} \longrightarrow \Delta J, \quad c_{xy} \longrightarrow \text{attitude model} \longrightarrow \hat{m},$$

with positive feedback and no cost term to arbitrate it. That loop was observed to lock onto self-consistent wrong solutions on light payloads.

3.7.2 Which residual identifies what

The two payload states enter through disjoint channels, which is what makes augmenting the second one nearly free in terms of sensing.

The composite mass enters the translational equation (3.10) as $1/m_T$ and is therefore identified predominantly by the *velocity* residual. It is not, however, absent from the rotational channel: it appears there through $c_{xy} = s/m_T$ in (3.26) and through the off-diagonal terms $B_i = (m_B/m_T)s_i r_z$ of (3.28), neither of which is frozen. What (3.34) removes is therefore not every mass dependence of $\hat{\omega}$, but the one that was demonstrably abused: the A term sits in the *denominator* $J_{xx} + A$ and carries the amplification $r_z^2/J_{xx} \approx 15$, so inflating $\hat{m}$ measurably suppresses an angular residual. The paths that remain are first order and unamplified — a relative error in m_T produces an equal relative error in

τ_{com} — and offer the optimiser no comparable lever.

The first moment s does not appear in (3.10) at all. It enters only through the trim moment (3.26) and the off-diagonal inertia (3.27), and is therefore identified exclusively by the *angular-rate* residual.

Both residuals are components of the same odometry measurement. The second state consequently costs no additional sensing — only the two extra decision variables and the solver time they imply, quantified in Chapter 5. Neither quantity is measured directly: both are recovered as the window-constant values that reconcile the reconstructed input u^{phys} with the observed v and ω trajectories.

It is worth recording why this separation was not available in the predecessor design. When the rotational channel is parameterised by (m_P, r_{xy}) with r_{xy} a fixed prior, the quantity the angular-rate residual actually identifies is the product $m_P r_{xy}$ — so an error in the assumed r_{xy} is converted directly into a mass error. Numerically, on the eccentric-hover condition, halving the assumed lever arm biased the mass estimate by 14.4% and setting it to zero by 41.6%, whereas a $\pm 33\%$ error in r_z changed the mass estimate by 0.00%. Augmenting the product removes the first sensitivity entirely; the second never existed, which is why r_z may safely remain a weak prior. The insensitivity is structural rather than fortunate: r_z is annihilated by the cross product in (3.13), so the torque channel carries no information about it in the first place and no amount of excitation would identify it. A Cramér–Rao bound computed for the eccentric-hover window tells the same story from the other side: with the rotational channel carrying no mass information the bound is $\sigma_m = 0.0109\text{kg}$, and with the channels coupled it is $\sigma_m = 0.0011\text{kg}$, with 94% of the information arriving through rotation.

3.7.3 Prior-free initialisation

The estimator requires an initial iterate for the mass channel. Supplying the calibrated airframe mass would be a prior about the answer; the implementation instead constructs the seed from measurements.

The simplest form is the hover approximation obtained by setting $a_z = 0$ and $\cos \theta = 1$ in (3.11),

$$m^{(0)} = \text{clip}\left(\frac{T_{\text{phys}}}{g}, \underline{m}, \overline{m}\right), \quad (3.42)$$

which is prior-free because T_{phys} comes from (3.36) and involves no model state. Its weakness is that it is only valid near hover: on a 4 ms^{-1} figure-eight the instantaneous ratio T_{phys}/g swings from -42% to 30% of the true mass, because the collective thrust is also supplying centripetal acceleration and is tilted away from the vertical.

The deployed seed therefore uses the whole window rather than one sample. Summing the exact vertical balance (3.11) over the N stages of the window,

$$m_T \sum_{i=0}^{N-1} (g + a_{z,i}) = \sum_{i=0}^{N-1} T_i \cos \theta_i, \quad (3.43)$$

and eliminating the accelerations by the telescoping identity

$$\sum_{i=0}^{N-1} a_{z,i} = \sum_{i=0}^{N-1} \frac{v_{z,i+1} - v_{z,i}}{\Delta t} = \frac{v_{z,N} - v_{z,0}}{\Delta t}, \quad (3.44)$$

gives a seed that requires no numerical differentiation of the velocity signal at all:

$$m^{(0)} = \frac{\sum_{i=0}^{N-1} T_i \cos \theta_i}{Ng + \frac{v_{z,N} - v_{z,0}}{\Delta t}}. \quad (3.45)$$

Only the two velocity endpoints of the window enter, so the estimate is immune to the sample-to-sample noise that a finite-difference acceleration would amplify; the tilt factor $\cos \theta_i$ from (3.4) restores the vertical component that

(3.42) discards; and every input is either T_{phys} or odometry, so the seed remains decoupled from the estimator's own state. The implementation falls back to (3.42), and then to a neutral box value, when the window is not yet full or the denominator of (3.45) degenerates.

Initial guess versus prior. The distinction matters for the claims of this dissertation and is easy to blur. Equation (3.45) enters the solver in two places: the initial iterate of the SQP, and the mean of the arrival cost. The former affects only the number of iterations, never the optimum. The latter affects the optimum in proportion to its weight, which is deliberately set so low that the corresponding standard deviation is several kilograms — far wider than the range of masses the vehicle can carry. The seed therefore influences where the optimiser starts, not where it converges. The same expression is used on re-initialisation after repeated solver failure, for the same reason: restarting from a measurement is safer than restarting from a possibly corrupted previous estimate.

3.8 Control: How the Parameters Re-centre the Cost

The nonlinear model predictive controller carries the parameters as runtime values,

$$p_{\text{NMPC}} = [x_{\text{ref}}(13); m_T; \Delta J; c_{xy}(2); d(3); \xi(3)] \in \mathbb{R}^{23}, \quad (3.46)$$

where d and ξ are lumped translational and rotational disturbances used only by the $\mathcal{L}_1$ comparison baseline [20] and identically zero under the proposed

method. Over a prediction horizon of N stages it solves

$$\begin{aligned}
 \min_{u_{0:N-1}} \quad & \sum_{k=0}^{N-1} \left(\|e_{\text{track}}(x_k, x_{\text{ref},k})\|_Q^2 + \|u_k - u_{\text{hover}}(p)\|_R^2 \right) \\
 & + \|e_{\text{track}}(x_N, x_{\text{ref},N})\|_P^2 \\
 \text{s.t.} \quad & x_{k+1} = F(x_k, u_k; p_{\text{NMPC}}), \\
 & \underline{T} \leq T_k \leq \bar{T}, \quad |\tau_{k,i}| \leq \bar{\tau}_i,
 \end{aligned} \tag{3.47}$$

applying only u_0 and re-solving at the next step, with the thrust box taken from the actuator envelope (3.39).

3.8.1 Why the input baseline must track the parameters

The second term of (3.47) penalises the deviation of the input from a *baseline* u_{hover} , and that baseline is recomputed from the live parameters at every solve rather than fixed when the solver is generated:

$$T_h = m_T(g - d_z), \quad u_{\text{hover}} = [T_h, c_y T_h, -c_x T_h, 0]^\top. \tag{3.48}$$

Consider first why a baseline is needed at all. Penalising $\|u\|_R^2$ outright would treat the thrust required merely to remain airborne as an expense to be minimised, biasing the solution toward insufficient thrust. The baseline recentres the penalty on the input that sustains equilibrium, so that R penalises only genuine control effort.

Consider next what happens when the baseline is *stale*. Suppose u_{hover} is computed from the unloaded mass while the vehicle carries a parcel. The cost then pulls the thrust persistently toward a value below what equilibrium requires, and the optimum settles where the marginal tracking cost balances the marginal baseline-deviation cost — at a nonzero steady-state error. The behaviour has a

counter-intuitive signature that identifies it in practice: *increasing R makes the error worse*, because R is precisely the strength with which the incorrect baseline pulls. No amount of weight tuning removes a bias of this kind; only correcting the baseline does. The effect is not marginal — with a heavy load and an empty-airframe baseline the tracking error was observed to stall above 0.85 m without converging.

The torque entries of (3.48) are exactly $-\tau_{\text{com}}$ from (3.13), and they exist for the same reason. Hovering with an eccentric load requires a constant trim torque (Section 3.3.3); a cost that does not admit this pulls the torque toward zero and produces the same class of steady-state bias in the attitude channel. The two equations are complementary and neither suffices alone: (3.13) teaches the *model* that the moment exists, while (3.48) teaches the *objective* that it is necessary. The d_z term in T_h plays the identical role for the lumped-disturbance baseline: with d in the model, vertical equilibrium is $T/m_T - g + d_z = 0$, so omitting d_z from the penalty centre would reintroduce the very bias the baseline exists to remove.

3.8.2 Inner-loop bandwidth

Model consistency in the outer loop cannot restore inner-loop bandwidth. The rate controller running on the flight controller is tuned for the bare airframe; a payload that multiplies the roll and pitch inertia lowers its effective bandwidth, and for heavy eccentric loads the resulting attitude oscillation grows until the torque saturates. The relevant ratio is

$$\kappa = \frac{J_{xx} + \Delta J}{J_{xx}}, \quad (3.49)$$

which (3.22) puts at 5.30 for the heaviest payload considered: that payload sits at the ceiling of what inner-loop retuning alone can absorb on this airframe. Together with the torque budget (3.23), this defines a feasibility boundary on

payload mass and eccentricity that is characterised experimentally in Chapter 5.

The predecessor implementation applied κ by rewriting the flight controller’s rate-loop gains at attachment. The deployed implementation does not touch those parameters; it scales the body-rate *command* instead, which keeps the adaptation inside the ROS layer, makes it continuous, and makes it reversible. The mechanism is derived in Section 4.6.

3.9 What is Estimated, What is Derived, and What is Not

Table 3.2 records the identity of every parameter in the model. Two scalars are online degrees of freedom; everything else is either derived from them by the algebra of this chapter, calibrated once for the airframe, or retained as a weak prior whose error has been shown not to matter.

Three design choices in that table invite objection, and each is answered by an argument already made above.

Why not augment the inertia increment as well? Because it is nearly unobservable in the flight regime of interest. The increment acts only through $\dot{\omega}$ in (3.12), and a delivery task — spent largely in near-hover, low-rate flight — excites that channel weakly. Admitting ΔJ as a free variable would let the optimiser explain measurement noise with a quantity the data barely constrains. There is a second, sharper reason: J_T appears *inverted* in (3.12) and is propagated N steps forward, so an error in it is amplified rather than damped, whereas the additive lumped term ξ of (3.46) — the alternative used by disturbance-based methods — is always safe. Estimating six inertia elements would also add decision variables whose natural scale is 10^{-2} , two orders of

Table 3.2: Model parameters, classified by identity. Only the composite mass and the two components of the horizontal first mass moment are decision variables of the estimator; the inertia increment and the CoM offset are generated from them.

Symbol	Meaning	Dim.	Identity	Enters through
m_T	composite mass	1	MHE decision variable	(3.10), (3.48)
s	horizontal first mass moment	2	MHE decision variable	(3.26), (3.27)
c_{xy}	CoM horizontal offset	2	derived, s/m_T	(3.13), (3.48)
ΔJ	roll/pitch inertia increment	1	derived, A of (3.31)	(3.12), (3.49)
m_P	payload mass	1	derived, $(m_T - m_B)^+$	(3.31)
r_z	payload vertical arm	1	weak prior, -0.47 m	(3.31), (3.27)
r_{xy}	payload horizontal arm	2	not used	—
m_B	bare-airframe mass	1	calibration	2.0643 kg
J_B	bare-airframe inertia	3	calibration	diag(0.0142, 0.0142, 0.0210)
k_f, k_m	thrust and yaw-reaction coefficients	2	calibration	8.55×10^{-6} , 0.016
k_d, g	drag coefficient, gravity	2	constants	0.05, 9.81

magnitude below the mass, to a Hessian already sensitive to scaling.

Why not augment the horizontal arm? Because nothing observes it. Every appearance of r_{xy} in the equations of motion is inside the product $m_P r_{xy}$, which is the state s we do augment. Estimating the factors separately would add an unidentifiable direction, and would reintroduce the release singularity of Section 3.5.1.

Why is a prior on r_z acceptable when a prior on r_{xy} is not? Because their error sensitivities differ by orders of magnitude. An error in r_{xy} propagates directly into the mass estimate through the product the rotational residual identifies; an error in r_z enters only A and the off-diagonal terms, both of which are second-order effects in near-hover flight where $\dot{\omega} \approx 0$. A $\pm 33\%$ perturbation of r_z moved the mass estimate by less than the reporting precision.

The information boundary. One mass-valued constant does remain in the closed loop, and it is disclosed rather than absorbed: the platform's 0.30 kg payload *envelope*. It is used for the attach-time inertia bootstrap of Section 4.6.3, for the physical admissibility bound on s in Section 4.4.4, and for the envelope treatment of Table 3.1. It is a specification of the airframe, in the same category as J_B or the actuator envelope (3.39), and it is not the mass of the parcel being carried on any particular flight. The precise claim of this dissertation is therefore: *no task-specific payload mass, and no attach or release notification, enters the estimator or the controller's model update*; it is not the stronger claim that the flight stack contains no payload-scale constant whatsoever.

3.10 Declared Modelling Approximations

Four approximations are stated explicitly here rather than left to be discovered.

The second-order horizontal inertia terms are discarded. The group C_{ij} of (3.28) is set to zero, as derived in Section 3.5.3. The error is (3.32), 2.6 % of the roll/pitch increment at the platform geometry, and the reason is the degeneracy (3.30). This is a deliberate exchange of a small bias for a well-posed release limit.

Anisotropy of the inertia increment is not represented. The deployed model (3.31) carries a single ΔJ on both roll and pitch, whereas (3.21) gives $\Delta J_{xx} \neq \Delta J_{yy}$ whenever $r_x \neq r_y$. After the C_{ij} terms are discarded the two become identical, $\Delta J_{xx} = \Delta J_{yy} = A$, so this approximation is subsumed by the previous one rather than additional to it.

The yaw increment is neglected. The third expression in (3.21), $\Delta J_{zz} = \mu(r_x^2 + r_y^2)$, is not added to J_{zz} . At 0.3 kg and $r_{xy} = 0.109\text{m}$ it amounts to 0.0031kgm^2 , or 14.8 % of J_{zz} . This is a known and non-negligible approximation. It is retained because in the deployed coordinates ΔJ_{zz} belongs to the C_{ij} group and would reintroduce (3.30), and because the yaw channel neither carries payload compensation nor participates in any release decision.

The vertical CoM offset is absent by construction. As shown in Section 3.3.3, c_z drops out of (3.13) as an exact algebraic consequence of collinearity, not as a truncation. It is listed among the approximations only to forestall the opposite reading.

The inertia is treated as constant in the body frame. Equation (3.12) omits the $\dot{J}_T \omega$ term that a time-varying inertia would contribute. The omission is exact on either side of a grasp or release and wrong only across one, for the duration of the transfer itself. Within the estimator the affected samples fall inside a single window and are absorbed as process noise; no attempt is made to model the transfer dynamically.

Two further simplifications are inherited from the rotor model rather than the rigid-body model. The thrust coefficient k_f is a static constant, and is known to lose validity under large induced inflow during fast translation [34]; and the yaw reaction of (3.36) is the steady-state term only, omitting the rotor angular-acceleration contribution $\sum_i J_r \dot{\omega}_i$, which is negligible in hover and mild manoeuvre but not during rapid yaw. Both are re-examined in Chapter 5, where the sensitivity of the complete interface to a mis-calibrated k_f is measured directly.

Chapter 4

The Eventless MHE–NMPC Framework

Chapter 3 established *what* is estimated and why. This chapter states *how*: the numerical form of the two optimal control problems, the interface that carries the estimate from one to the other, and the logic that decides when the controller may believe that its payload has gone.

The organising principle of the chapter is stated once here and applied throughout. *No attach or release notification is evidence*. The controller issues the gripper command itself, and still does not use it to update its model. The only thing that clears a payload from the model is persistent estimator evidence. Section 4.4.1 argues why a system that possesses the command should nevertheless refuse to use it; the remainder of the chapter builds the machinery that makes the refusal affordable.

4.1 Closed-Loop Architecture

Figure 4.1 shows the information flow. Three layers are involved: the physics and flight stack (Gazebo, PX4 software-in-the-loop, and a magnetic detachable-joint gripper that produces genuine inertia and centre-of-mass changes); the inner rate-control loop on the autopilot; and a ROS outer loop containing the

estimator and the predictive controller.

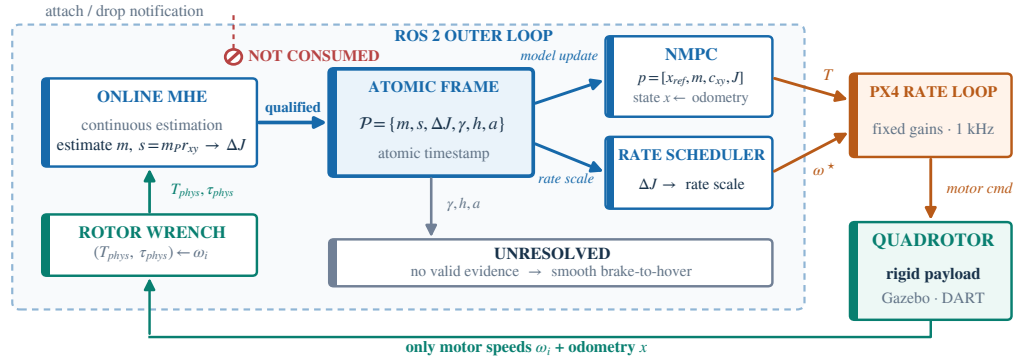


Figure 4.1: System architecture and information flow. The complete pipeline consumes only rotor speeds and odometry. One atomic, health-gated $m/s/J$ frame drives both the NMPC model and the continuous body-rate command scheduling. No attach or release notification enters this path.

Two properties of this diagram carry the argument of the dissertation.

First, **the only inputs are odometry and rotor speeds**. The estimator does not read the controller’s intended thrust or torque (Section 3.6), does not read the gripper’s attach or release record, and does not read the simulator’s joint state. The payload truth available in simulation is used exclusively for offline scoring.

Second, **there is exactly one interface**. The estimator publishes a single atomic frame (Section 4.3) containing mass, first moment, the derived geometry, a confidence scalar and a health flag. Both consumers — the NMPC model parameters and the inner-loop command scaling — read that one frame. An earlier design in which mass, geometry and inertia travelled on separate topics, each with its own latency and validity, made it possible for the controller to hold a mutually inconsistent parameter set; the atomic frame makes that class of fault unrepresentable.

The two optimisers run at different rates and do not share a solve clock. The estimator solves at 10Hz over a 2s window; the controller solves at 20Hz over a 1s preview; setpoints are published at 50Hz by interpolating along the most recent predicted trajectory.

4.2 Moving Horizon Estimator

4.2.1 The optimisation

With the augmented state (3.40) of Chapter 3, the estimator solves, at each 10 Hz tick,

$$\begin{aligned}
 \min_{x_{0:N}, w_{0:N-1}} \quad & \|x_0 - \bar{x}_0\|_{W_0}^2 + \sum_{j=0}^{N-1} \beta_j \left(\|y_j - Cx_j\|_{W_y}^2 + \|w_j\|_{W_w}^2 \right) \\
 \text{s.t.} \quad & x_{j+1} = F_M(x_j, u_j^{\text{phys}}) + Gw_j, \\
 & \underline{m} \leq m_T \leq \bar{m}, \quad |s_x|, |s_y| \leq \bar{s},
 \end{aligned} \tag{4.1}$$

where y_j is the odometry measurement, C selects the thirteen vehicle states from the sixteen-dimensional augmented state, F_M is one explicit fourth-order Runge–Kutta step of the dynamics of Chapter 3 driven by the reconstructed wrench u^{phys} of (3.36), and

$$G = \begin{bmatrix} I_{13} \\ 0_{3 \times 13} \end{bmatrix} \tag{4.2}$$

confines the process noise to the physical states, leaving m_T and s noise-free within the window.

The horizon is $N = 20$ stages at $\Delta t = 0.1$ s. The choice is a compromise that the experiments of Chapter 5 revisit: a longer window averages more measurement noise, but the constraint that makes the payload states identifiable — that they are *constant* over the window — becomes less defensible the longer the arc of trajectory the window spans. On a 4 ms^{-1} figure-eight a ten-stage window recovered the mass $3.7\times$ more accurately than a twenty-stage one, while a forty-stage window sat on the lower bound for the entire flight.

4.2.2 Weights

All three weight matrices follow Bryson’s rule: each diagonal entry is the inverse square of a representative magnitude for the corresponding residual, so that every channel contributes a cost of order unity at its own natural scale and the Gauss–Newton Hessian is not curvature-imbalanced across directions.

The measurement weight uses representative state-estimation standard deviations,

$$W_y = \text{diag}\left(\sigma_p^{-2}I_3, \sigma_v^{-2}I_3, \sigma_q^{-2}I_4, \sigma_\omega^{-2}I_3\right), \quad (4.3)$$

with $\sigma_p = 0.02\text{ m}$, $\sigma_v = 0.05\text{ m s}^{-1}$, $\sigma_q = 0.01$ (about 1.1°) and $\sigma_\omega = 0.02\text{ rad s}^{-1}$.

The process-noise weight is deliberately *tighter* than the measurement weight,

$$W_w = \text{diag}(10^4I_6, 10^5I_4, 10^3I_3). \quad (4.4)$$

This is the central tuning decision of the estimator, and its rationale is worth stating explicitly. The purpose of (4.1) is not to smooth noisy measurements; it is to force any persistent mismatch between the reconstructed wrench and the observed motion to be explained by the two payload parameters rather than absorbed into unstructured process noise. Trusting the dynamics — large W_w — is what makes that attribution happen. The angular block is one decade looser than the translational block because the rotational equation contains the gyroscopic product $\boldsymbol{\omega} \times J_T \boldsymbol{\omega}$ and the inverse inertia, and is numerically the more delicate of the two.

The arrival cost anchors the start of the window,

$$W_0 = \text{diag}(10^3I_6, 10^4I_4, 10^2I_3, \sigma_{m,0}^{-2}, \sigma_{s,0}^{-2}I_2), \quad \sigma_{m,0} = 3.16\text{ kg}, \quad \sigma_{s,0} = 0.1\text{ kg m}, \quad (4.5)$$

with $\bar{x}_0$ taken from the previous window’s solution shifted forward by one stage. Two points about the payload entries. First, they are four to five orders of

magnitude softer than the physical-state entries: with $\sigma_{m,0} = 3.16\text{kg}$ against a vehicle that weighs 2.06kg empty, the mass anchor carries essentially no information, and that is intentional — it is what allows a payload step to be absorbed within roughly one window rather than being dragged toward the previous answer. Second, they are not set to zero. A zero entry removes positive definiteness of the Hessian in that direction whenever a window happens to be poorly excited, and an earlier experiment with genuinely uninformative payload channels produced 472 solver failures from exactly that mechanism. A numerically small but non-zero anchor is uninformative in the statistical sense while remaining well posed in the numerical one.

The bounds in (4.1) are physical rather than tuning knobs. The composite mass is constrained to $[0.95m_B, 5.0\text{kg}] = [1.961\text{kg}, 5.0\text{kg}]$: the vehicle cannot weigh less than its own airframe, and the 5% margin leaves room for calibration error rather than pinning the estimate at the calibrated value. That such a bound is a *hard constraint on a decision variable* is a concrete advantage of the moving-horizon formulation over a recursive filter, which can be biased away from non-physical estimates but cannot be forbidden from producing them. The first moment is bounded loosely, $\bar{s} = 1.5\text{kgm}$, purely to prevent the optimiser from diverging; the physically meaningful admissibility limit on s is applied later, in the reference-formation logic of Section 4.4.4, where it can reject a sample without distorting the optimisation.

The prior-free ablation. The lower bound $0.95m_B$ and the arrival mean are the two remaining places where the airframe mass could in principle inform the mass channel. The implementation therefore retains a switch that removes both: the bound drops to 0.2kg — a value chosen only to keep $1/m_T$ away from its singularity and containing no airframe information — and $\sigma_{m,0}$ rises to 1000kg . This configuration is an *ablation*, not the deployed default; the results of Chapter 5 are reported under the default unless stated otherwise. The seed of Section 3.7.3 is prior-free in both configurations, because it is computed from

measurements in either case.

4.2.3 Stage weighting: what was tried and what is deployed

The factors β_j in (4.1) exist because the constant-payload model is valid on either side of a grasp or release but not across one. When the event falls inside the current window, a fixed-weight estimator returns a compromise between two physical regimes until the older regime leaves the horizon — a transient on the order of the window length.

Three treatments were implemented and compared.

Event-triggered de-weighting (retired). The original scheme detected a persistent step in T_{phys} and scaled every stage preceding it by 10^{-4} , leaving the arrival cost active so that the physical-state trajectory remained available as a warm start. On the 2 ms^{-1} condition this schedule remained armed but never fired. At 4 ms^{-1} it was *disabled*, because grasp and release transients are not the only events that move the collective thrust: a trajectory-mode change produces a step of the same amplitude, and the detector cannot distinguish them. All results reported in Chapter 5 use $\beta_j \equiv 1$. A parameterised generalisation of the schedule was additionally optimised by cross-entropy search, and found no measurable improvement over the hand-designed rule; that negative result is substantive, and is discussed in Section 5.10.4.

Exponential forgetting (available, off by default). A continuous alternative discounts each stage by its age,

$$\beta_j = \lambda^{N-1-j}, \quad j = 0 \text{ oldest}, \quad (4.6)$$

giving an effective memory of about $1/(1-\lambda)$ stages — ten frames (1.0s) at $\lambda = 0.9$, five at $\lambda = 0.8$. This is the smooth version of shortening the horizon: it neither changes the problem dimension nor introduces a new noise channel, and the arrival-cost block is explicitly excluded from the discount, since that block anchors physical states rather than carrying stale payload evidence. The deployed default is $\lambda = 1$.

Parameter random walk (available, off by default). The third option admits process noise on the payload channels themselves, replacing $\dot{m}_T = 0$, $\dot{s} = 0$ by a random walk with tuned intensities $\sigma_{\dot{m}}$ and $\sigma_{\dot{s}}$, as is standard in the learned-MHE literature [23]. It removes the need for any event logic, since the parameters can then move within a window. It is off by default because the added noise channels have natural scales two orders of magnitude apart from the physical ones, and the resulting Hessian conditioning produced the 472 solver failures noted above.

The deployed configuration therefore uses no event logic at all. This is a deliberate simplification and it costs a transient of roughly one window after each payload change; the lifecycle logic of Section 4.4 is built to tolerate that transient rather than to shorten it.

4.2.4 Re-anchoring and over-range rejection

Two guards protect the estimator from its own failures.

Re-anchoring after repeated solver failure. After five consecutive failed solves the window is discarded and rebuilt: the physical states are reset to the current measurement, the mass is re-seeded from (3.45), and s is reset to zero. The choice of zero is not neutral — it asserts “no payload” — and it therefore has

a consequence that the release logic must respect: a first moment that is zero *because the window was just rebuilt* is not evidence of a release. Section 4.4.5 states that rule explicitly.

Physically inadmissible first moments. The platform cannot realise a first moment larger than

$$\|s\|_{\max} = m_P^{\text{env}} r_{xy}^{\max} = 0.30 \text{ kg} \times 0.13 \text{ m} = 0.039 \text{ kg m}, \quad (4.7)$$

so a solved $\|\hat{s}\|$ above that bound is a statement about the estimator, not about the payload. Such samples are excluded from the reference-formation window of Section 4.4.4 and counted as an estimator-quality diagnostic; if they persist while the vehicle is not manoeuvring, the window is re-anchored. They are *not* removed from the published frame, which is a limitation recorded in Chapter 5: an out-of-envelope $\hat{s}$ still reaches the controller’s c_{xy} through the filter and slew limiter.

4.3 The Continuous Payload Interface

4.3.1 One atomic frame

After each solve the estimator publishes

$$\mathcal{P} = \left\{ m_T, s_x, s_y, c_x, c_y, \Delta J_{xx}, \Delta J_{yy}, \Delta J_{zz}, J_{xx}, J_{yy}, J_{zz}, \gamma, h, a \right\}, \quad (4.8)$$

in which the geometry entries are computed from (m_T, s) by (3.25) and (3.31), $\gamma \in [0, 1]$ is the no-payload confidence of Section 4.4.3, h is a health bit and a is the age of the most recent successful solution.

Health is a conjunction: the latest solve must have succeeded, and the odom-

etry, rotor-speed and known-input streams must all be fresher than 0.30 s. The consumer applies one further test of its own, on the receive age of the frame itself (0.35 s), because a frame can be healthy at the moment it is produced and stale by the time it is used.

The asymmetry in how an unhealthy frame is treated is deliberate and is the safety property of the interface: *an unhealthy or stale frame freezes the last accepted model and resets the confidence persistence timer*. A degraded estimate can therefore delay a release confirmation, but it can never cause one.

4.3.2 Filtering and slew limiting

A valid frame is not consumed directly. Each channel passes a first-order low-pass filter and a separate rate limiter before it reaches the solver:

$$\alpha = 1 - e^{-\Delta t/\tau}, \quad \hat{\theta} \leftarrow \hat{\theta} + \alpha(\theta^{\text{tgt}} - \hat{\theta}), \quad \hat{\dot{\theta}} \leftarrow \hat{\dot{\theta}} + \text{clip}(\cdot, \pm \dot{\theta}_{\max} \Delta t), \quad (4.9)$$

with $\tau = 0.20$ s on the controller side (0.25 s on the estimator’s own output stage) and rate limits 0.60 kg s^{-1} for the mass, $0.080 \text{ kg m s}^{-1}$ for the first moment and $0.080 \text{ kg m}^2 \text{ s}^{-1}$ for the inertia increment.

The reason for a limiter in addition to a filter is that the two protect against different things. The filter attenuates noise; the limiter bounds how fast the *model* underneath a receding-horizon optimisation may change. A parameter step between consecutive solves invalidates the warm start, and an optimiser that must re-converge from scratch at 20 Hz is an optimiser that occasionally does not converge in time. The limiter converts any step — including a genuine one at grasp — into a ramp the solver can track.

4.4 The Eventless Grasp–Release Lifecycle

4.4.1 Why refuse the command

The controller in this work issues the gripper-release command itself. It would be trivial to clear the payload model at that instant, and every command-triggered implementation does so. The reason not to is that a release command is a *request*, and the model must describe the *vehicle*.

The two diverge in practice. A gripper can be slow to open, can jam, can release only part of a multi-item load, or can release nothing at all; conversely, a load can be torn off, can slip, or can be released by an agent whose notification never arrives. In each case a command-triggered controller flies a loaded vehicle with an empty model, or an empty vehicle with a loaded one. The failure is silent, because nothing in the control loop contradicts it.

The eventless alternative accepts a cost — confirmation is delayed by the time the estimator needs to accumulate evidence — in exchange for a model that is never wrong for a reason the vehicle cannot detect. Chapter 5 measures both sides of that trade.

4.4.2 Two state machines

The lifecycle is split between the two nodes, as shown in Figure 4.2. The estimator owns payload *presence* and may *declare* a release. The controller owns the payload *model* and clears it only on *confirmation*. Declaration is neither necessary nor sufficient for confirmation; the separation exists so that a single detector cannot, by itself, empty the controller’s model.

Presence is a hysteretic $\text{EMPTY} \leftrightarrow \text{LOADED}$ machine driven by sustained payload-

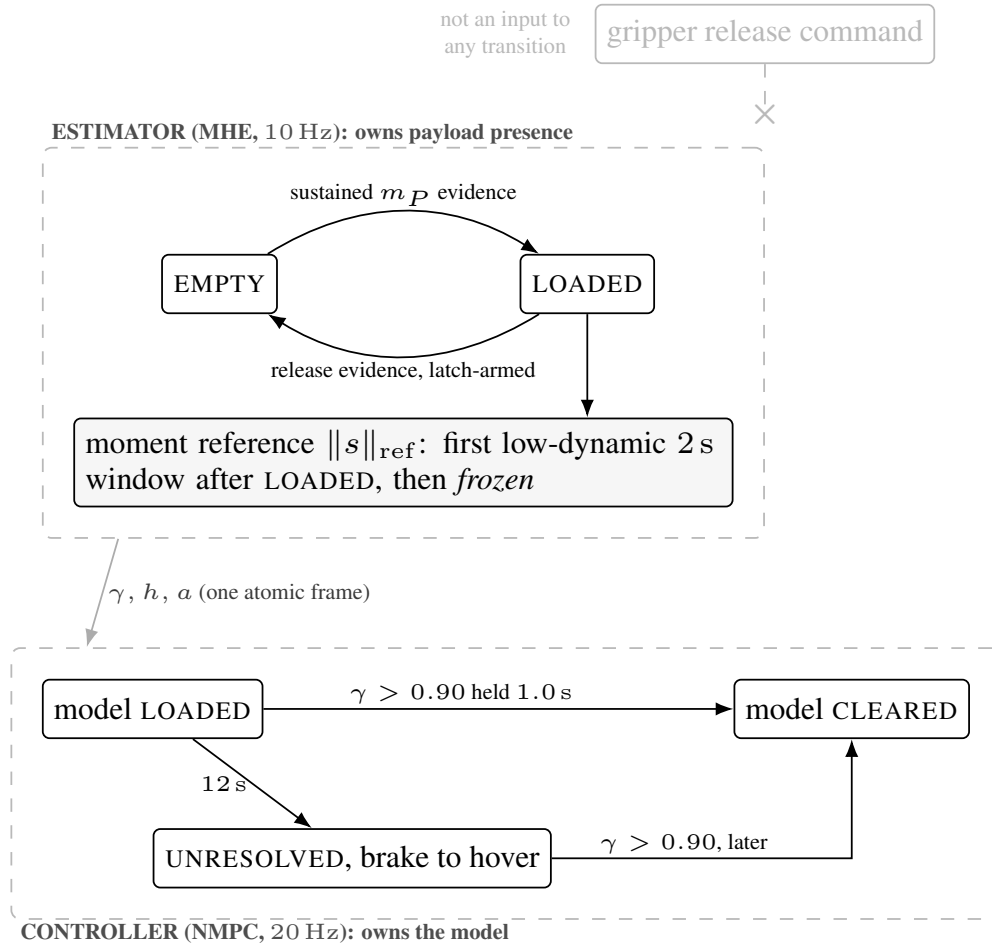


Figure 4.2: The payload lifecycle as two state machines. The estimator owns payload presence and may declare release from the evidence of (4.15); the controller owns the model and clears it only on confirmation, $\gamma > 0.90$ held for 1.0 s. The gripper release command enters neither machine. UNRESOLVED is a hold, not a clearing: the model stays loaded while the vehicle brakes to hover and keeps waiting for evidence.

mass evidence, with a directional thrust residual as a fast path into LOADED. A separate latch records that the present flight reliably entered LOADED at some point. That latch does two jobs: it prevents an empty-flight confidence value from consuming the later release confirmation, and it prevents a transient mass dip during the figure-eight from disarming the true release path.

4.4.3 No-payload confidence

Every lifecycle decision is a function of the published frame (4.8) alone. The confidence scalar γ fuses three “empty” indications.

Let $\sigma(v; a, b)$ be the smooth indicator that is 1 for $|v| \leq a$, 0 for $|v| \geq b$, and in between

$$\sigma(v; a, b) = 1 - (3\chi^2 - 2\chi^3), \quad \chi = \frac{|v| - a}{b - a}, \quad (4.10)$$

which is the standard cubic smooth-step: continuous with continuous first derivative at both ends, so γ cannot jump when a channel crosses a threshold. Then

$$\gamma = \sigma(m_P; 0.015, 0.060) \cdot \sigma(\|s\|; 0.0015, 0.0060) \cdot \sigma(\Delta J_{xy}; 0.0020, 0.0100), \quad (4.11)$$

with thresholds in kg, kg m and kg m², and $\Delta J_{xy} = \max(|\Delta J_{xx}|, |\Delta J_{yy}|)$.

The choice of a *product* rather than a sum or a vote is deliberate: any single payload indication vetoes EMPTY. In particular a non-zero first moment keeps γ low even when the scalar mass estimate happens to be sitting on its lower bound — which, as Section 3.5.1 showed, is exactly the state the predecessor design entered after a release. The product turns that failure mode into a refusal to confirm rather than a false confirmation.

It is worth being precise that (4.11) is a *state-quality* metric for a payload that has already gone, not a transition detector. The moment factor carries an absolute threshold, and a state transition must not depend on the estimator’s noise floor happening to lie below an absolute number. The release detector of Section 4.4.5 therefore uses the individual indications, not their product.

4.4.4 The loaded moment reference

The release detector is built on a *ratio*,

$$\rho = \frac{\|s\|}{\|s\|_{\text{ref}}}, \quad (4.12)$$

and the reason is empirical. Over five development flights the loaded and released distributions of $\|s\|$ *overlap*: the loaded minimum, 0.0039 kgm, lies below the released maximum, 0.0049 kgm. No absolute threshold separates them. The ratio does, because it compares the vehicle against itself.

That requires a reference the estimator must form autonomously. A running maximum is not acceptable — it would absorb precisely the figure-eight estimation errors the detector exists to reject, and 7.6 % to 18.3 % of loaded frames exceed the physical bound (4.7) during the manoeuvre, which would depress the denominator until ρ crossed its threshold with the payload still attached. The deployed procedure instead forms a reference once and freezes it:

1. **Admissibility.** After the LOADED declaration, a healthy frame is admitted only if $\|\hat{s}\| \leq 0.039 \text{ kgm}$ by (4.7). An out-of-envelope sample is discarded, not clipped: clipping would still build the denominator from a sample known to be wrong.
2. **Low dynamics.** The vehicle must satisfy $\|\omega\| \leq 0.15 \text{ rad s}^{-1}$ and $\|v_{xy}\| \leq 0.20 \text{ m s}^{-1}$. This is an *online* criterion, not “wait N seconds after grasp”: the latter would only be correct for one particular mission script.
3. **Persistence.** Twenty consecutive admitted frames (2 s) must accumulate; any manoeuvre clears the buffer.
4. **Magnitude stability.** The window must satisfy

$$\frac{1.4826 \cdot \text{MAD}(\|s_j\|)}{\text{median}(\|s_j\|)} \leq 0.25, \quad (4.13)$$

where the factor 1.4826 converts the median absolute deviation into a consistent estimate of the standard deviation for Gaussian data. A robust dispersion measure is used rather than the sample standard deviation because a single outlying frame must not be able to veto an otherwise clean window.

5. **Direction consistency.** The window must further satisfy

$$\left\| \frac{1}{20} \sum_j \frac{\mathbf{s}_j}{\|\mathbf{s}_j\|} \right\| \geq 0.90, \quad (4.14)$$

that is, the mean *unit* first-moment vector must be close to unit length. This catches the failure mode that (4.13) cannot: a sequence whose magnitude is perfectly steady while its direction rotates or flips sign frame to frame, which is the signature of an ill-conditioned rotational channel rather than of a real eccentric load.

6. **Robust statistic and freeze.** The reference is the 20 %-trimmed mean of the window magnitudes, and it is frozen for the remainder of the flight.

If no window qualifies before the vehicle enters the aggressive segment, ρ never becomes available and the moment-gated path cannot fire. That is a genuine miss mode, and it is the dominant one in the results of Chapter 5.

4.4.5 Release evidence and the decision rule

The detector organises its inputs into three *independent* information sources. The word is used strictly: an earlier version counted the mass channel and the inertia channel as two votes, which was false robustness, since in the continuous interface $\Delta J = A(m_P)$ is a deterministic function of the mass and the fixed r_z . They carry one piece of evidence, not two.

$$\begin{aligned}
 \text{moment collapse} & : \rho < 0.30 \wedge \|s\| < 0.008 \text{kg m}, \\
 \text{quantity empty} & : m_P < 0.030 \text{kg}, \\
 \text{residual drop} & : \text{a signed thrust step, Section 4.4.6.}
 \end{aligned} \tag{4.15}$$

The absolute bound in the first line is a sanity limit, not a discriminator — the distributions overlap, as noted above — and it exists only to reject the case in which an anomalously large reference peak makes ρ spuriously small.

The decision rule is then

$$\text{release} \iff \text{armed} \wedge [\text{slow path} \vee \text{fast path}], \tag{4.16}$$

in which

$$\begin{aligned}
 \text{slow path} & : \text{quantity empty} \wedge \text{moment collapse, sustained 5 frames,} \\
 \text{fast path} & : \text{strong residual} \wedge \text{moment collapse, sustained 4 frames,}
 \end{aligned} \tag{4.17}$$

where *armed* records that the present flight reliably entered LOADED at some earlier point.

Moment evidence is mandatory in both paths, and the reason is a recorded failure. An earlier rule admitted a third path — residual together with quantity-empty, without moment evidence — and it released prematurely in 3 of 9 randomised flights, the earliest 51.1 s before the release command. Its design assumption, that “quantity empty is false while the payload is attached”, is simply untrue at 4ms^{-1} : the mass estimate sits on its lower bound for whole stretches of a figure-eight, so m_P is persistently below 0.03 kg for a median of 14 s and as long as 48 s in loaded flight. Neither amplitude nor persistence separates that from a real release. The plain residual vote is likewise false-positive during manoeuvre. The discarded path was therefore two unreliable channels conjoined, with nothing able to veto them. The moment channel is the one that measured cleanly: across those same flights the loaded ρ bottomed at 0.789 against a

threshold of 0.30, a factor of 2.6 of margin, with zero false positives.

The cost of making moment evidence mandatory is accepted explicitly: *a release whose ρ never collapses is not detected here*. That is a miss, and it is handled by the UNRESOLVED path of Section 4.4.7 — never by releasing on weaker evidence.

Two further rules close known loopholes. First, moment evidence must come from a fresh, successful solve: the re-anchoring of Section 4.2.4 initialises s to zero, and that zero is not counted as a collapse. Second, the online and offline evaluations call the same pure decision function, so a replay of a recorded flight exercises exactly the deployed logic rather than a re-implementation of it.

4.4.6 The release-residual vote

The residual channel compares two adjacent half-windows of the reconstructed collective thrust,

$$\Delta T_k = \bar{T}_{\text{post}} - \bar{T}_{\text{pre}} = \frac{1}{h} \sum_{i=k-h+1}^k T_i^{\text{phys}} - \frac{1}{h} \sum_{i=k-2h+1}^{k-h} T_i^{\text{phys}}, \quad h = 3, \quad (4.18)$$

that is, two 0.3 s means at 10 Hz. A release makes ΔT negative, because the vehicle no longer supports the load.

Amplitude alone is not enough: a recorded figure-eight transient reached -1.69 N, which is larger in magnitude than the -1.47 N weight of the smallest payload tested. The vote therefore combines a floor with a robust significance test against the recent history of ΔT itself,

$$\hat{\sigma}_{\Delta T} = \max\left(1.4826 \cdot \text{MAD}(\{\Delta T_i\}_{i < k}), 0.05 \text{ N}\right), \quad z_k = \frac{-\Delta T_k}{\hat{\sigma}_{\Delta T}}, \quad (4.19)$$

where the history deliberately excludes the current frame, since a step included

in its own dispersion estimate inflates $\hat{\sigma}$ and hides itself. A vote is cast when $\Delta T_k < -0.9N$ persists for two frames, and a *strong* vote — the one eligible to pair with a moment collapse in (4.16) — when it persists for four.

Residual votes are *edge* events and are given an explicit validity window of 3 s. A vote that has expired is no longer evidence. This is a small detail with a large consequence: without it, a single manoeuvre transient early in a flight would remain available to be combined with an unrelated moment collapse minutes later.

4.4.7 Confirmation, UNRESOLVED, and braking to hover

The controller clears its payload model through exactly one route:

$$\gamma > 0.90 \quad \text{held for } 1.0\text{s} \quad \text{on healthy, fresh frames.} \quad (4.20)$$

An estimator declaration accelerates this but does not bypass it. On accepted release evidence the estimator switches only the *published target* of s to zero; the optimisation state is not overwritten. The output filter of (4.9) consequently produces a continuous decay rather than a geometric step, and γ rises smoothly through (4.11). Confirmation can also occur with no declaration at all, driven by the estimator converging on its own; the moment factor in (4.11) still vetoes it while $\|s\|$ is non-zero.

If no confirmation arrives within 12 s of the release command, the controller enters UNRESOLVED. This state is the reason the eventless design is deployable, and its semantics deserve emphasis: **a timeout is not a release**. The model stays loaded. What changes is the vehicle’s exposure: it stops flying an aggressive trajectory it may no longer have the authority to fly, and it gives the estimator a quiet operating point in which Section 4.4.4’s low-dynamic criterion can finally be met.

The braking manoeuvre is a finite-time reference rather than an instantaneous switch to hover, because at 4 ms^{-1} the latter would command a step of several metres per second. Let $\mathbf{p}_0, \mathbf{v}_0$ be the state at the declaration and $T_b = 3 \text{ s}$. The stopping point is chosen as the displacement of a constant deceleration from $\mathbf{v}_0$ to rest,

$$\mathbf{p}_1 = \mathbf{p}_0 + \frac{1}{2} T_b \mathbf{v}_0, \quad (4.21)$$

and the reference between the two is the unique quintic satisfying $\mathbf{p}(0) = \mathbf{p}_0$, $\dot{\mathbf{p}}(0) = \mathbf{v}_0$, $\ddot{\mathbf{p}}(0) = \mathbf{0}$, $\mathbf{p}(T_b) = \mathbf{p}_1$, $\dot{\mathbf{p}}(T_b) = \mathbf{0}$, $\ddot{\mathbf{p}}(T_b) = \mathbf{0}$:

$$\begin{aligned} a_3 &= (10\Delta\mathbf{p} - 6T_b\mathbf{v}_0)/T_b^3, \\ \mathbf{p}(t) &= \mathbf{p}_0 + \mathbf{v}_0 t + a_3 t^3 + a_4 t^4 + a_5 t^5, \quad a_4 = (-15\Delta\mathbf{p} + 8T_b\mathbf{v}_0)/T_b^4, \quad (4.22) \\ a_5 &= (6\Delta\mathbf{p} - 3T_b\mathbf{v}_0)/T_b^5, \end{aligned}$$

with $\Delta\mathbf{p} = \mathbf{p}_1 - \mathbf{p}_0$. Zero acceleration at both ends means the manoeuvre begins and ends without a jump in commanded thrust, which is what keeps the transition from itself perturbing the estimator whose evidence is being awaited. Attitude is blended over the same interval by the smooth-step of (4.10).

4.4.8 Threshold provenance

Table 4.1 lists every lifecycle threshold and where its value came from. All were fixed in code before the final campaign began and were not retuned on it. Only the ratio threshold and the residual floor were selected against data, and that data is from earlier flights that are disjoint from the reported campaign; the remainder are engineering choices whose closed-loop sensitivity has not been swept, and are reported as such.

Table 4.1: Lifecycle thresholds and their provenance. “Earlier flights” predate, and are disjoint from, the reported campaign.

Threshold	Value	Provenance
moment-collapse ratio ρ	0.30	leave-one-out cross-validation on 59 earlier flights; every fold selected 0.30, whereas 0.20 succeeded in only 48 of 59
residual floor / persistence	0.9 N; 2 or 4 frames	below the 1.47 N weight of the smallest payload; on a recorded flight, figure-eight excursions past 0.9 N lasted at most three frames while the release step lasted four
slow-path persistence	5 frames (0.5 s)	engineering choice
mass-domain release	$m_P < 0.030$ kg, 20 frames	between the loaded minimum (0.079 kg) and the post-release maximum (0.007 kg) of earlier 0.15 kg flights
no-payload confidence	0.90 held 1.0 s	engineering choice
reference window	20 frames; CV $\leq$ 0.25; direction $\geq$ 0.90	engineering choice
UNRESOLVED timeout / brake	12 s / 3 s	engineering choice

4.5 Adaptive NMPC

4.5.1 Cost and weights

The controller solves (3.47) with $N = 20$ stages at $\Delta t = 0.05$ s, preserving a 1.0 s preview while running at 20 Hz. The residual vector is

$$y_k = \begin{bmatrix} e_p; & e_v; & e_{q,\text{vec}}; & e_\omega; \\ & & & u_k - u_{\text{hover}}(p) \end{bmatrix} \in \mathbb{R}^{16}, \quad (4.23)$$

where the attitude entry is the vector part of the relative quaternion $\mathbf{q}_{\text{ref}}^* \otimes \mathbf{q}$, which keeps the residual on the tangent space of the rotation manifold instead of treating four quaternion components as independent Euclidean coordinates.

Weights again follow Bryson’s rule, $Q_{ii} = L_i^{-2}$, with tolerances

$$L_p = 0.1 \text{ m}, \quad L_v = 0.3 \text{ ms}^{-1}, \quad L_q = 0.05, \quad L_\omega = 0.3 \text{ rad s}^{-1}, \quad (4.24)$$

and, on the inputs, $L_T = 2\text{N}$, $L_{\tau,xy} = 0.5\text{Nm}$, $L_{\tau,z} = 0.2\text{Nm}$; the torque scales are taken to be the constraint bounds themselves, since the input cannot exceed them in any case. The terminal weight rescales the stage blocks by 20 (position), 1 (velocity), 2 (attitude) and 0.2 (rate): the endpoint of the prediction is pulled tightly onto the reference in position and attitude, while relaxing the terminal rate demonstrably helps the solver.

This non-dimensionalisation is not cosmetic. Before it was applied, position and torque weights differed by a factor of tens at their natural magnitudes, the Gauss–Newton Hessian was correspondingly ill-conditioned across directions, and the stationarity residual was observed to reach 6.26×10^5 — a complete numerical breakdown rather than a slow convergence.

Trajectory-scaled tolerances. Two of the entries in (4.24) are defined by the comment that accompanies them as trajectory-dependent quantities: L_v is a characteristic speed and L_ω a characteristic yaw rate. Written as constants they are correct only for the low-speed reference they were tuned on. An optional rescaling restores the intent,

$$L_v \leftarrow \max(L_v, r w), \quad L_\omega \leftarrow \max(L_\omega, K_\psi w), \quad K_\psi = 3.17, \quad (4.25)$$

taking the maximum so that the weights are only ever relaxed, never tightened. The constant K_ψ is the ratio of peak yaw rate to angular rate on the figure-eight of Section 4.7, obtained numerically; it is worth noting that evaluating the yaw rate at the intuitive location $a = \pi/4$ gives 2.83 and underestimates the peak by 12 %, because the extremum does not occur there. Applying (4.25) at the low-speed condition reproduces L_v exactly but relaxes L_ω by a factor of 3.2, which exposes a pre-existing mismatch rather than introducing one: the original value was documented as matching a *circular* trajectory, whose yaw rate is constant at w , while the vehicle actually flies a figure-eight whose peak is $3.17w$. The rescaling is off by default so that historical batches remain re-

producible.

4.5.2 Solver

Both optimal control problems are transcribed by direct multiple shooting with explicit fourth-order Runge–Kutta integration, a Gauss–Newton Hessian approximation, and partial-condensing HPIPM [10] as the quadratic subproblem solver, generated and solved with acados [8]. Both use *full* sequential quadratic programming with merit-function backtracking rather than the real-time-iteration scheme [6]: RTI was observed to diverge silently in this configuration, and a controller that reports success while producing a diverging input is worse than one that takes an extra millisecond. Generated C code is cached and reused until the model or problem dimensions change.

4.6 Inner-Loop Scheduling

4.6.1 From inertia ratio to a command scale

Section 3.8.2 established that a five-fold inertia increase costs the autopilot’s rate loop most of its bandwidth, and that the outer loop cannot recover it by being more accurate. The deployed response scales the body-rate *command* around the current measured rate,

$$\omega'_{\text{cmd}} = \omega + \varsigma (\omega_{\text{cmd}} - \omega), \quad \varsigma \geq 1, \quad (4.26)$$

applied to the roll and pitch channels only, since ΔJ enters only J_{xx} and J_{yy} . Equation (4.26) exaggerates the rate *error* the inner loop sees, which is equivalent to raising its proportional gain, without modifying any autopilot parameter. Because it is a common scale on the roll/pitch pair it preserves the direction

of the rate-error vector.

The requested scale is the inertia ratio (3.49), clipped at a cap of 5.0, evaluated from the live ΔJ of the payload frame. Three filters stand between the request and its application, each addressing a distinct failure:

- a **dead band** of 0.08 on the target, so that estimator noise cannot move it, with explicit snapping to 1.0 and to the cap so that hysteresis cannot strand the empty-airframe steady state permanently above unity;
- a **first-order filter** with $\tau = 0.5\text{s}$, chosen far slower than the inner loop (hundreds of hertz) and still fast enough to follow the second-scale progressive load transfer during a grasp;
- **asymmetric rate limits**, 4.0s^{-1} rising against 1.5s^{-1} recovering, encoding the asymmetry of the consequences: being late to add authority risks saturation, being late to remove it does not.

4.6.2 Headroom limiting

Scaling a command that is already close to the rate limit accomplishes nothing except saturation, at which point the scaling has silently stopped acting. The implementation therefore solves for the largest scale that keeps the scaled command inside a guard band $\pm L$, with L equal to 95% of the hard rate limit $\bar{\omega} = 2.0\text{rad s}^{-1}$.

For one channel, write the rate error $e = \omega_{\text{cmd}} - \omega$. Requiring $|\omega + \varsigma e| \leq L$ gives

$$\varsigma \leq \begin{cases} \frac{L - \omega}{e}, & e > 0, \\ \frac{-L - \omega}{e}, & e < 0, \end{cases} \quad (4.27)$$

and the applied scale is the smallest such bound over the roll and pitch channels, floored at unity:

$$\zeta_{\text{eff}} = \max\left(1, \min_{i \in \{x,y\}} \zeta_i^{\max}\right). \quad (4.28)$$

The floor at unity guarantees that the scheduler can never make the raw controller command *less* authoritative than it would have been without scheduling — the scheduler is allowed to help and not allowed to hurt. When (4.28) binds, it is logged, so that saturation avoidance is an observable event rather than a silent degradation.

4.6.3 Attach-time inertia bootstrap

One interval resists the eventless principle: the second or so between a load beginning to take weight and the estimator accumulating enough evidence to say so. During that interval the vehicle is physically loaded and the model is not, and it is the interval in which the predecessor implementation lost eleven of sixteen flights at the heaviest condition.

The deployed bootstrap addresses it with the narrowest intervention that works. At the start of lift, the *inertia* parameter alone is floored at the value implied by the platform’s 0.30 kg payload envelope; mass and first moment are untouched, and no payload is fabricated in either. The overlay is then faded out over 0.5 s once the estimator independently reports persistent loaded evidence, namely $\Delta J \geq 0.010 \text{ kg m}^2$ together with a confidence well below the confirmation threshold, sustained for 0.5 s. If the estimate becomes stale or unhealthy while the handover is in progress, the fade freezes at the last safe inertia rather than continuing toward zero.

Three properties keep this consistent with the rest of the design. It is *inertia-only*, so it cannot influence the mass or moment channels on which every release decision depends. It is *one-directional*, active only at attach, so the release

path remains estimate-only. And its magnitude is the airframe’s payload envelope, which Section 3.9 has already disclosed as a platform specification rather than task information.

4.7 Reference Trajectory

The evaluation trajectory is a three-dimensional Geroni lemniscate,

$$x_r = r \sin a, \quad y_r = r \sin a \cos a = \frac{r}{2} \sin 2a, \quad z_r = z_h + d_z \sin a, \quad a = wt_c, \quad (4.29)$$

with yaw commanded along the velocity vector. This curve is chosen because its self-intersection is well behaved: differentiating (4.29) gives $v = rw(\cos a, \cos 2a)$, whose magnitude at the crossing $a \in \{0, \pi\}$ is $\sqrt{2}rw \neq 0$, while both acceleration components are proportional to $\sin a$ or $\sin 2a$ and vanish there. The velocity therefore does not reverse and the curvature does not blow up — it is not a cusp — yet the yaw must turn substantially faster through the crossing than on a circle of the same speed, which is precisely the coupling the evaluation wants. The vertical term gives the two lobes different heights, increasing coupling while keeping the workspace compact.

Amplitude is grown from zero after an initial hover by the quintic smooth step $\alpha(s) = 10s^3 - 15s^4 + 6s^5$, which has zero first and second derivative at both ends. The ramp duration is not arbitrary. The velocity during the ramp is $\dot{\alpha}(x_f - x_0) + \alpha \dot{x}_f$; requiring the extra term contributed by the amplitude growth to stay below half the trajectory’s own characteristic speed gives

$$\frac{1.875r}{t_{\text{ramp}}} \leq \frac{1}{2}\sqrt{2}rw \quad \implies \quad t_{\text{ramp}} \geq \frac{2.65}{w}, \quad (4.30)$$

where $1.875/t_{\text{ramp}}$ is the peak of $\dot{\alpha}$. The radius cancels: how long a ramp is needed depends only on the angular rate, not on the size of the figure.

4.8 Online Execution Sequence

At every control instant the implementation performs the following sequence.

1. Receive and frame-convert the latest odometry.
2. Reconstruct the physical wrench from rotor speeds, (3.36), averaging the 250 Hz samples over the estimation interval.
3. Append the measurement and the reconstructed input to the 10 Hz window; re-seed by (3.45) if the window must be rebuilt.
4. Solve (4.1) for (m_T, s) and the physical trajectory.
5. Update the presence machine, the frozen moment reference (Section 4.4.4) and the residual vote (Section 4.4.6); evaluate the release decision (4.16).
6. Assemble and publish the atomic frame (4.8) with its health bit and solution age.
7. In the controller: validate freshness, low-pass filter and slew-limit the frame (4.9), and write m_T , ΔJ and $c_{xy} = s/m_T$ into the model parameters and the hover baseline (3.48).
8. Evaluate confirmation (4.20) and the UNRESOLVED timeout; switch the reference to the braking quintic (4.22) if the timeout expires.
9. Shift the previous solution forward as a warm start, solve (3.47), and publish the first input; invert thrust to throttle by (3.38) and apply the body-rate scaling (4.26)–(4.28).
10. Log estimates, residuals, constraint activity, solver statistics and every life-cycle transition.

Estimator and controller failures are handled separately and asymmetrically. An invalid estimator solve retains the last accepted parameter set and marks the

frame unhealthy, which — by Section 4.3.1 — freezes the controller’s model and resets its confirmation timer. An invalid controller solve falls back to the configured safe action rather than publishing an unconstrained command.

Chapter 5

Implementation and Experimental Evaluation

5.1 Software-in-the-Loop Platform

The implementation is organised as ROS 2 packages around PX4 software-in-the-loop [35] and Gazebo [36]. The controller node owns the reference generator, the predictive optimal control problem, the mission sequence and the release-confirmation logic; the estimator node owns the sliding measurement window, the physical-input reconstruction, the payload-state optimisation, the lifecycle detector and the published payload frame. The two communicate through the single atomic frame of Section 4.3.

One platform decision is worth recording because it constrains what can be claimed. Earlier work in this project used a Gazebo plugin that changed a link’s inertial properties at run time, which would have allowed a mass step without a second body. That mechanism does not work: the simulator’s component is written and readable, but the physics engine continues to use the value from the model description, so the vehicle never actually changes mass. Every payload change reported here is therefore produced by the detachable-joint gripper, which attaches and removes a genuine second rigid body and consequently couples mass, inertia and centre of mass together. The retired mass-step line

Table 5.1: Main implementation components of the deployed pipeline.

Component	Role	Principal signals
Gazebo (Harmonic)	Rigid-body physics; detachable-joint gripper	pose, rotor speed, attach/detach record (logging only)
PX4 SITL	State estimation, rate loop, control allocation	odometry, actuator commands
mhe_node	16-state moving-horizon estimation; lifecycle detector	odometry, rotor speed $\rightarrow$ payload frame (4.8)
acados_nmpc_node	Predictive control, mission sequence, confirmation	payload frame, state $\rightarrow$ thrust and body-rate setpoint
payload_estimate	Frame schema, confidence (4.11), release decision (4.16)	shared by the online node and the offline replay
proximity_gripper_node	Grasp contact and release actuation	proximity, grasp enable, release command
lifecycle_trace	Machine-readable event trace for scoring	every state transition, with timestamps
l1_adaptive	Comparison adaptive controller	lumped matched disturbance d, ξ
px4_pid_benchmark	Bare-autopilot cascade baseline	position setpoints only

has been removed from the repository, and no wrench-emulated result is retained.

5.2 Platform Parameters

The reusable-vehicle constants are

$$m_B = 2.0643 \text{ kg}, \quad J_B = \text{diag}(0.0142, 0.0142, 0.0210) \text{ kg m}^2, \quad k_d = 0.05, \quad (5.1)$$

with rotor arm 0.174 m, $k_f = 8.55 \times 10^{-6} \text{ N s}^2$ and $k_m = 0.016 \text{ m}$. The collective-thrust box is the actuator envelope (3.39), $[1.27 \text{ N}, 31.35 \text{ N}]$, and the torque bounds are 0.5 Nm in roll and pitch and 0.2 Nm in yaw. None of these grows with the estimate.

The value m_B appears in the model, in the derived quantity $m_P = (m_T - m_B)^+$ and in offline scoring. Payload masses of 0.15 kg and 0.30 kg and grasp offsets up to 0.15 m are used in the experiments; none of these values is supplied to the estimator or the controller.

The task sequence is: approach under position control; hand-off to the predictive controller and warm-up hover; controlled descent; grasp enable; lift; figure-eight carry; phase-triggered release; recovery. The scope of the controller is stated explicitly — it governs the grasp hover and the post-event adaptation phase, and the approach controller is not part of any claim. Batches run headless.

5.3 Experiment Design

The evaluation separates four questions: whether the payload states are identifiable at all; whether the eventless interface completes a release; whether refusing the command buys anything; and where the interface fails.

5.3.1 Release-baseline arms

The model-clearing logic is parameterised, so that the arms differ in exactly one factor wherever possible. Table 5.2 lists them.

Arm A additionally uses the legacy, non-continuous geometry path and schedules the inner loop by rewriting autopilot rate gains, whereas arm B uses the continuous frame and body-rate command scaling. The arms therefore differ *before* release as well as at it, and per-arm flight outcomes are reported separately rather than pooled.

Table 5.2: Release baselines implemented as switchable arms. The estimator and the controller each implement their half of the same named setting.

Arm	Name	Model-clearing rule
B	Eventless-Full	proposed: estimator evidence only; command never consulted
A	CMD-Immediate	clears payload geometry and inner-loop scaling at the release command, with no confirmation
C	CMD-Evidence	the proposed detector, but persistence counted only after the command arrives
—	P-NoMoment	the proposed interface with the moment-gated path removed (mass domain only)
—	Oracle	clears at the true separation instant; an upper bound, not deployable

5.3.2 Conditions and scenarios

Three nominal conditions are used: 0.15 kg at 2 ms^{-1} , 0.15 kg at 4 ms^{-1} and 0.30 kg at 4 ms^{-1} . The last is deliberately a stress test at the vehicle’s actuator limit rather than an operating point, for reasons quantified in Section 5.11.

Five scenarios are run on top of those conditions:

1. **Nominal grasp–carry–release**, paired arm A against arm B.
2. **Delayed detachment**. A fault injector accepts the normal release command but holds the physical joint for a median 4.01 s. This separates “the command was issued” from “the payload left”.
3. **Unsignaled loss**. The payload is detached at the release phase with no command and no notification, emulating a gripper that opens by itself or a load torn off.
4. **Partial loss**. Four 0.05 kg boxes are grasped together and three are removed one at a time, at 45, 80 and 115 s after grasp, with no signal; a control arm carries the same boxes without loss.
5. **Thrust-map mismatch**. The estimator’s thrust coefficient alone is scaled by $\pm 5 \%$, leaving the plant and the throttle inversion untouched. This em-

ulates a mis-calibrated k_f on real hardware, which Section 5.11 identifies as the leading transfer risk.

Eligibility is declared before scoring: a flight enters the release statistics if the payload physically attached, according to the gripper’s own attach record, and the vehicle survived to the release command. This differs from the pre-registered rule, which judged attachment from the estimator’s mass evidence and therefore silently excluded flights whose estimator failed to recognise an attached payload — exactly the cases most worth reporting. Both denominators are given wherever they differ.

5.4 Metrics

Position tracking is summarised by

$$\text{RMSE}_p = \sqrt{\frac{1}{K} \sum_{k=1}^K \|p_k - p_k^{\text{ref}}\|_2^2}. \quad (5.2)$$

Estimator metrics are the composite-mass error, the first-moment error against an offline reconstruction of truth, the fraction of samples resting on a bound, and the number of window re-anchoring. Lifecycle metrics are the completion rate, the confirmation delay measured from the release command to the end of the 1.0 s confidence hold, the confirmation path taken, and the count of declarations occurring before any command or injected loss. Computational metrics are the median and upper percentiles of the solve time for both optimisers.

Ground truth is used only in offline scoring. For the first moment, truth is reconstructed after the fact as m_{pr_y} from the simulator-measured grasp offset; it never reaches the estimator, which runs with no horizontal attachment geometry at all.

5.5 Real-Time Performance

On one desktop core shared with the simulator and the autopilot, the 13-state controller ($N = 20$, 20 Hz) solves in 1.3 ms at the median and 2.2 ms at the 99th percentile over $n = 1009$ solves: 2.6 % and 4.4 % of its 50 ms period.

The 16-state estimator ($N = 20$, 10 Hz) solves in 9.2 ms at the median, 24.6 ms at the 90th and 59.2 ms at the 99th percentile, with a 72.8 ms maximum over $n = 3740$ solves (the first twenty frames of each flight discarded), against a 100 ms period. This is roughly seven times the 1.4 ms median of the historical mass-only estimator: augmenting the first moment costs no additional sensing, as Chapter 3 argued, but it is not computationally free.

The deadline is met in simulation and the tail margin is narrow. We therefore make *no* claim about headroom on an onboard computer; that requires measurement on the target hardware.

5.6 Estimation of the Payload States

Figure 5.1 shows the first-moment estimate against reconstructed truth over eleven 0.15 kg flights at 2 ms^{-1} . The qualitative behaviour is the one the design intends: $\hat{s}_y$ is zero before grasp, rises with no attach notification, holds near truth through the figure-eight, and decays to zero after release rather than stepping.

Two observations qualify the figure. First, the carry-phase estimate carries a bias toward zero whose cause has not been identified; this is one reason the detector is built on the ratio (4.12) rather than on an absolute magnitude, since a common scale factor cancels in the ratio. Second, the s_x (pitch) channel is consistently noisier than the s_y (roll) channel, which matches the torque-

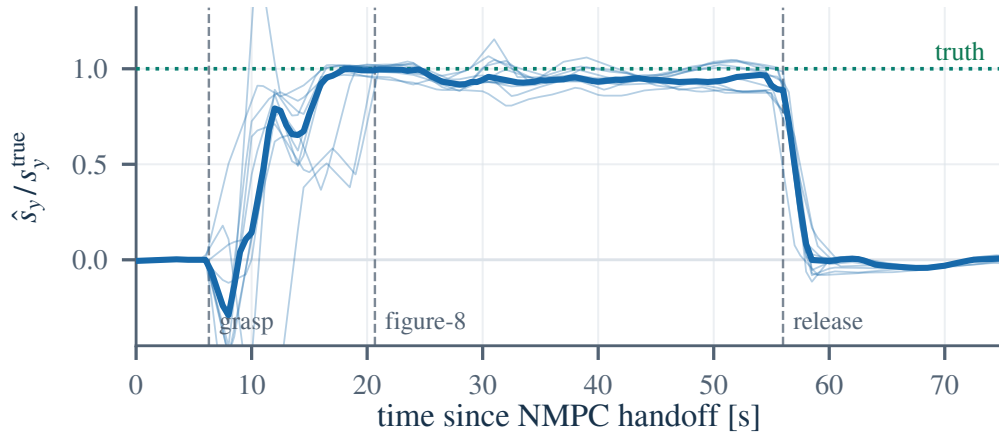


Figure 5.1: First mass moment against ground truth (0.15 kg at 2 ms^{-1} , $n = 11$, each flight normalised by its own truth). Truth is reconstructed offline from the simulator-measured grasp offset and never reaches the estimator. The carry-phase estimate is biased toward zero in the batch shown; the cause is not identified. The moment-gated detector uses the ratio ρ of (4.12), which is insensitive to a common scale bias.

reconstruction correlations reported in Section 3.6.

Figure 5.2 shows two complete missions in three dimensions, including the two distinct post-release behaviours that Section 5.7 quantifies.

5.7 Completion and Confirmation Delay

The complete interface confirmed release in 32 of 36 eligible flights (Table 5.3); the two-sided 95 % Clopper–Pearson interval is $[73.9\%, 96.9\%]$. All four failures occurred at 4 ms^{-1} and all four trace to estimator failure rather than to the decision logic: three never recognised the payload and remained UNRESOLVED, and one diverged after release. The pre-registered eligibility rule would have excluded the first three and reported 31/32; the rule adopted here retains them.

The confirmation delay is *bimodal*, and the two modes correspond to the two paths of Section 4.4.7 rather than to a spread within one mechanism. In 17 of the 32 completed flights the confidence crosses its threshold while the vehicle

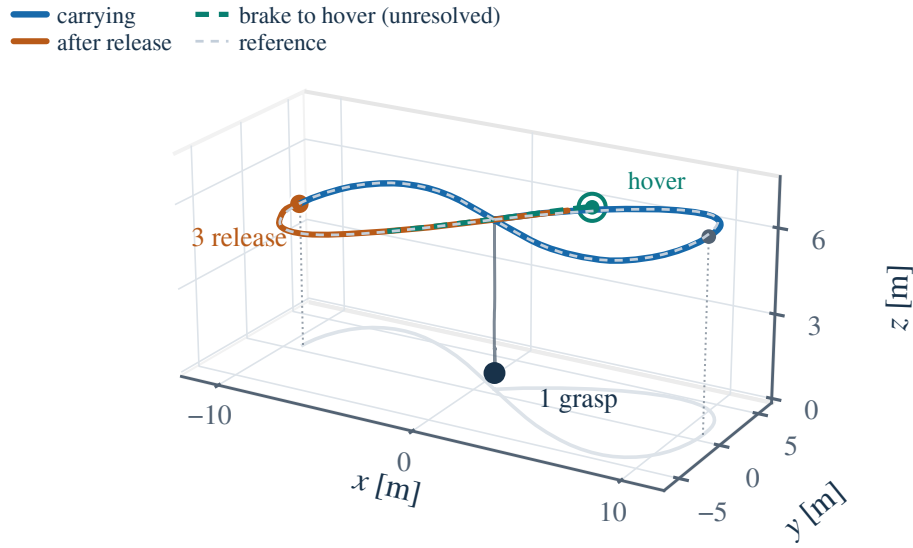


Figure 5.2: Measured mission trajectory (simulator ground truth), two $0.15 \text{ kg}/2 \text{ ms}^{-1}$ flights of the same batch: grasp, lift, then carry along the figure-eight with $\pm 0.8 \text{ m}$ vertical undulation against the nominal reference. After the release command one flight clears the moment vote in manoeuvre and keeps tracking; the other declares UNRESOLVED at its 12 s timeout and brakes to hover, confirming at 16.40 s. The two paths agree to within 0.12 m (median) before the timeout. Loaded tracking RMS is 0.053 m and 0.060 m after release.

is still flying the figure-eight, and confirmation completes in a median of 3.46 s. In the remaining 15, no release evidence is available in manoeuvre — either $\hat{s}$ has drifted under sustained excitation, or the frozen reference of Section 4.4.4 never formed — so the system reaches its 12 s timeout, declares UNRESOLVED, brakes to hover over 3 s, and confirms at a median of 16.56 s. Pooling the two would describe neither.

Figure 5.3 shows one complete in-manoeuve confirmation.

Table 5.4 records which route produced each confirmation. Every confirmation used the controller’s confidence route (4.20); the columns record the estimator declaration, where one occurred. That three distinct routes appear is the reason the completion rate must be read as a result for the *interface as a whole* rather than as a validation of any single detector.

No flight in any path declared release before the command. In 23 of the 32

Table 5.3: Release completion and confirmation delay, all eligible arm B flights. Delay runs from the release command to the end of the 1.0 s confidence hold and is computed over completed flights. The two delay columns are distinct control paths and are not pooled.

condition	n	compl.	delay median [IQR]	
			in manoeuvre	via brake-to-hover
0.15 kg, 4 ms^{-1}	11	10/11	2.90 s [2.90, 2.90]	16.60 s [16.60, 16.60]
0.30 kg, 4 ms^{-1}	13	10/13	8.95 s [8.00, 9.26]	16.40 s [16.20, 16.70]
0.15 kg, 2 ms^{-1}	12	12/12	3.31 s [2.95, 4.36]	16.48 s [16.40, 16.50]
pooled	36	32/36	3.46 s [2.90, 8.00]	16.56 s [16.46, 16.60]

Overall delay maximum 16.70 s. Share confirming only after braking to hover: 50 %, 40 %, 50 % of completed flights by row, 47 % pooled.

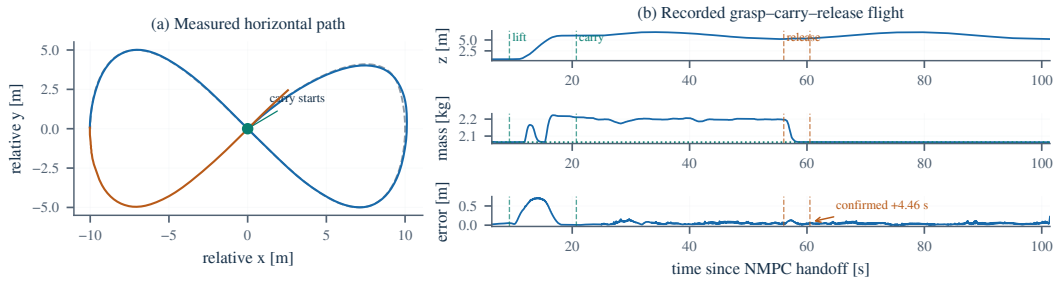


Figure 5.3: One eventless flight from the final batch (0.15 kg , 2 ms^{-1}). Left: measured horizontal path, coloured by carrying and post-release phase, against the nominal reference including its amplitude ramp. Right: height, the mass estimate actually consumed by the controller, and the position-error norm through grasp, lift, carry, release command and confirmation. This flight confirms in manoeuvre 4.46 s after the command; it illustrates one execution and does not replace the cohort statistics of Table 5.3.

completed flights the presence machine made exactly two transitions, one at grasp and one at release; in the nine confidence-only flights it never declared EMPTY within the logged window, which extended more than 40 s past confirmation. Every load declaration came from sustained mass evidence, and no external grasp signal was consumed.

Table 5.4: Confirmation paths in the 32 completed flights of Table 5.3. *Moment-gated*: the detector declared before the confidence crossed. *Mass-domain*: the frozen reference never formed, the confidence crossed during or shortly after the brake to hover, and the mass-domain route declared 0.9 s to 3.1 s later. *Confidence only*: the reference never formed and no declaration occurred.

condition	n	moment-gated	mass-domain	confidence only
0.15 kg, 4 ms ⁻¹	10	9	1	0
0.30 kg, 4 ms ⁻¹	10	1	3	6
0.15 kg, 2 ms ⁻¹	12	9	0	3
pooled	32	19	4	9

5.8 Does Refusing the Command Buy Anything?

5.8.1 Nominal pairs

Across 19 pairs in which both arms were eligible, both arms completed 19/19 releases with no observed post-release instability (Table 5.5). Arm A has zero confirmation delay by construction. Under nominal conditions, therefore, refusing the command buys nothing and costs a delay — which is the correct result, since nominal conditions are precisely those in which the command is true.

Table 5.5: Paired command-triggered ablation. Arm A clears the model at the command by construction; arm B requires estimator evidence; arm C is arm B with its release evidence armed only by the command. The last two rows inject a command/physical-detachment mismatch; “early” means model clearing before the joint is actually removed (“–”: not scored).

condition	pairs	A compl.	B compl.	early clear A/B
0.15 kg, 4 ms ⁻¹	8	8/8	8/8	–
0.30 kg, 4 ms ⁻¹	3	3/3	3/3	–
0.15 kg, 2 ms ⁻¹	8	8/8	8/8	–
nominal pooled	19	19/19	19/19	–
0.15 kg, 2 ms ⁻¹ + 4 s hold	5	5/5	5/5	5/5 / 0/5
same, arm C instead of A	6	6/6 (C)	6/6	0/6 / 0/6

Only three pairs formed at the 0.30 kg/4 ms⁻¹ condition, because arm A crashed during or immediately after lift in 11 of 16 attempts there. That failure is a property of arm A’s gain-rewriting schedule rather than of its release semantics:

its rate scaling starts from the lower-bound mass estimate while the box is still resting on the ground.

5.8.2 Delayed detachment

Under a median 4.01 s hold between command and physical detachment, the arms separate exactly as designed. Of twelve arm-runs one arm A run never armed the autopilot, leaving five valid pairs and one valid unpaired arm B run.

In all five pairs arm A cleared its payload model at the command, a median 4.01 s before the payload actually left. Arm B cleared before detachment in *none* of its six valid runs: it entered UNRESOLVED in 5/6 and confirmed a median 12.45 s after detachment, while the remaining run confirmed 3.00 s after detachment. All eleven valid runs carried an independent simulator detachment record.

The honest reading of this result is narrow. Median position-error peaks during the hold were 0.071 m for arm A and 0.068 m for arm B, so what the experiment demonstrates is a difference in *model–plant consistency*, not a tracking advantage and not crash avoidance. A permanently jammed gripper — under which arm A would fly a loaded vehicle with an empty model for the remainder of the mission — was not tested.

5.8.3 Command-armed confirmation and unsigaled loss

The delayed-detachment result reflects *persistent confirmation*, not eventlessness. Arm C demonstrates this directly: armed only by the command but otherwise identical to arm B, it also never cleared early (0/6 pairs) and confirmed sooner after detachment (median 4.3 s against 7.3 s). If the only hazard were a slow gripper, arm C would be the better design.

Eventlessness earns its keep when no release command reaches the controller at all. With such losses injected at the release phase, arm B detected the loss in 10 of 18 paired flights — 4/9 at 2 ms^{-1} , and 4/5 and 2/4 at 0.15 kg and 0.30 kg at 4 ms^{-1} , with a median latency of 2.2 s to 2.9 s — while arm C, never armed, detected none and carried a phantom first moment for the rest of the flight. Seven of the eight misses occurred because the loaded moment reference had not formed, or the payload had not been recognised, before the loss: the same mechanism that produces the brake-to-hover mode in Table 5.3.

The cost of running a release detector throughout loaded flight — the risk of a false positive while the payload is still attached — was measured and not observed. No release was declared before a command or an injected loss in 209 loaded flights spanning 3.7 h, giving a per-flight 95 % upper bound of 1.4 %.

5.9 Partial Payload Loss

Losing part of a load is the case that a binary attached/detached abstraction cannot represent at all, and it is where a continuous interface should show an advantage in kind rather than in degree.

Pre-registered criteria were: (a) $\hat{m}$ enters $\pm 0.02\text{ kg}$ of each new truth in at least 80 % of steps; (b) the paired tracking-RMS difference against the no-loss control arm is at most 0.02 m, with no crash; (c) no EMPTY declaration after the first two losses.

At 2 ms^{-1} all three criteria pass: the estimate followed 14 of 15 steps, with a median convergence of 1.1 s to 8.8 s, there was no false EMPTY declaration, and tracking was indistinguishable from the control arm. At 4 ms^{-1} criterion (a) reached only its threshold (12/15), so recovery is claimed at 2 ms^{-1} only.

5.10 Ablations

5.10.1 Thrust-map calibration

The mass channel recovers m_T from an *absolute* thrust, so it inherits the calibration error of k_f directly — nothing in the formulation cancels it. This ablation measures how much of that error the interface can absorb. The answer is: very little.

Scaling only the estimator’s thrust map by $\pm 5\%$ (0.15 kg, 2 ms^{-1} , four repetitions per arm, 11 valid flights) caused no pre-command release in any arm, but defeated the interface in both directions:

- at 0.95, the payload was never recognised (0/3) and every flight remained UNRESOLVED;
- at 1.05, a phantom 0.09 kg persisted after release (4/4), every flight remained UNRESOLVED, and one diverged;
- at the matched value 1.00, the payload was recognised in 4/4.

The interpretation is that k_f must be calibrated well inside 5% for this interface to function. That is a demanding requirement, and it is the single most important item for hardware transfer. Note also what the failure mode is: not a wrong release, but a refusal to confirm. The interface degrades toward UNRESOLVED rather than toward an empty model, which is the correct direction for a safety-relevant decision, but it is still a failure.

5.10.2 Window-balance re-initialisation

The window vertical-force seed of (3.45) replaced the instantaneous hover approximation (3.42). A pre-registered A/B comparison at the 4 ms^{-1} condition gives: valid flights 10/10 with the window seed against 8/10 without; figure-eight re-initialisations reduced from 29 to 1; seed error reduced from 17.1 % to 2.8 %; longest consecutive re-initialisation run reduced from 17 to 1; mission completion 10/10 with no post-release crash.

One reservation is recorded with the result: in the enabled arm only a single re-initialisation event occurred, so the mechanism was barely exercised there. The reported lifecycle cohort predates this change.

5.10.3 The inertia-coefficient mode

The four treatments of A listed in Table 3.1 behave as the analysis of Section 3.5.4 predicts. In synthetic grasp–release replay the live coefficient drives the post-release mass estimate up by 53 % and holds it there, whereas the lagged coefficient returns to -0.34% of the bare airframe, with s and ΔJ decaying together and no bound contact.

The envelope treatment separates the two requirements. It also removes the derivative, but pins the coefficient to the platform envelope rather than to the estimate: in closed loop it biased the loaded mass estimate to -3.58% , against 0.4% for the lagged form, and that bias was sufficient to make the mass-domain release criterion fire prematurely, 50 s before the true detachment. Breaking the derivative is necessary; retaining a correct value is a separate requirement, and pinning it to the wrong value is harmful.

5.10.4 A negative result on learned stage weights

The parameterised generalisation of the retired event-triggered schedule (Section 4.2.3) was optimised by cross-entropy search in the loop, and the resulting schedule showed no measurable benefit over the hand-designed rule in a paired ablation. The objective was flat over a broad neighbourhood of that rule.

This negative result is substantively useful, and is reported rather than omitted. What produced the benefit in the predecessor system was *noticing that the model had changed and removing stale data*; the precise magnitude of the post-trigger weights is weakly determined by the data. A learned schedule should therefore not be presented as a superior learned controller. It is also part of the reason the mainline retired event logic altogether.

5.11 Where the Interface Fails

5.11.1 Moment-reference formation

The dominant failure mode is not a wrong decision but an unavailable one. In 13 of the 32 completed flights the healthy, low-dynamic, in-envelope window of Section 4.4.4 never filled after the LOADED declaration, so the ratio test never ran at all. Those flights are the population of the brake-to-hover column of Table 5.3 and of seven of the eight unsignaled-loss misses.

A contributing factor was found and is smaller than the effect: the admissibility envelope (4.7) assumed a 0.13 m grasp offset while the gripper actually admitted 0.15 m, which affected 2 of 13 0.30 kg flights. It does not explain most of the missing references. Reliable reference formation therefore remains necessary work.

A related boundary is recorded honestly: out-of-envelope $\hat{s}$ samples are excluded from moment votes but still reach the controller's c_{xy} through the filter and slew limiter of (4.9). The exclusion protects the decision, not the model.

5.11.2 First-moment drift under sustained excitation

Sustained figure-eight excitation can raise $\|\hat{s}\|$ from 0.012 kg m to 0.031 kg m. Because moment evidence is then withheld, confirmation is delayed. Deceleration restored the vote within 3.3 s to 3.4 s in the affected flights, which is the mechanism behind the brake-to-hover delays.

An online counter of envelope exceedances correlates with this outcome at 4 ms^{-1} — in-manoeuve confirmation occurred in 9/10 flights with zero exceedances against 2/10 otherwise, Fisher exact $p = 0.006$ — but it misses all six delayed 2 ms^{-1} cases, so it is a partial diagnostic rather than a predictor. Exiting the manoeuvre earlier on that signal is unimplemented and unvalidated.

5.11.3 The operating limit

0.30 kg at 4 ms^{-1} is at the vehicle's actuator limit. Hover requires 74 % of maximum thrust, but it is torque, not thrust, that binds: in the figure-eight, pitch torque saturated within 98 % of five-second diagnostic windows and yaw within 53 % (46 % and 16 % at 0.15 kg, at most 6 % at 2 ms^{-1}), and the body-rate scaling of (4.26) ran at a median 4.39 of its cap of 5.0. This is the quantitative form of the feasibility boundary predicted by (3.23) and (3.49) in Chapter 3.

Three of the five arm B pre-release crashes and three of the four incomplete releases occurred at this condition. Every eligible 2 ms^{-1} flight confirmed its release. What is claimed at this limit is safe release *semantics*, not reliable

flight; deployment there needs torque margin.

5.11.4 Estimator failures

In four flights the published mass never left the empty value; in two of these, repeated solver failures caused re-initialisation from a lower-bound seed. The three at 4 ms^{-1} flew without diverging and remained UNRESOLVED.

One failure is more instructive than the rest. In a 0.30 kg flight the estimate was unhealthy for 48 s before release. Because the phase-timed release command is *not* gated on estimator health, the command was issued anyway; the frozen model no longer matched the vehicle after detachment, and the flight diverged. Gating release on estimator health is an obvious remedy and is not implemented.

5.11.5 Configuration and default-safety

Every reported release batch explicitly enables the release-evidence path. Its built-in default is *off*, under which every flight ends UNRESOLVED. Deployment therefore requires an explicit enable plus startup validation of the configuration, and the implementation includes a self-check that reports at startup which blockers, if any, would prevent the detector from ever firing.

5.11.6 Missing baselines

The only baselines reported here are the command-triggered and command-armed arms of Table 5.2, which isolate release semantics. The evaluation does *not* include a comparison against a fixed-parameter predictive controller, against a lumped-disturbance adaptive method such as $\mathcal{L}_1$ -adaptive NMPC [20]

or incremental nonlinear dynamic inversion [21], or against a change-detection procedure such as CUSUM [32] applied to the same signals under a matched false-alarm constraint. Comparison nodes for the first two exist in the implementation (Table 5.1) but no frozen result set is reported for them. This is the largest gap in the evaluation and is stated as such.

5.12 Threats to Validity and Reproducibility

All system-level evidence is software-in-the-loop. The leading threats are:

- **Thrust-map fidelity.** The plant generates thrust with the same square law the estimator inverts, and rotor speeds are exact. The $\pm 5\%$ experiment of Section 5.10.1 bounds one dimension of this; speed noise, transport delay and induced inflow [34] remain untested.
- **Clean state estimates.** The odometry available in simulation is better than a real vehicle's, and the measurement weights (4.3) are set from representative rather than measured noise levels.
- **Rigid attachment.** Gripper compliance and pendulum modes are not modelled. Neither affects which signals the estimator consumes, but both would add unmodelled dynamics to the rotational residual on which s depends.
- **Confounded disturbances.** A sustained vertical updraft or downdraft is intrinsically indistinguishable from a payload change on a collective-thrust-only residual. Wind is not evaluated in the reported cohort.

Reproducibility is handled by construction rather than by discipline. Every run records the software revision, the resolved value of every environment switch that affects behaviour, the problem dimensions, horizons, weights, bounds, event

timestamps, random seed and solver statistics. Runs terminated by numerical failure remain in the denominator. The release decision used online and the one used in offline replay are the same function, so a recorded flight can be re-scored without re-implementing the logic.

Table 5.6: Attempts and exclusions in the paired campaign. Because the arms differ before release, these rates are reported per arm rather than pooled, and neither characterises end-to-end mission reliability.

Arm	Attempts	Disposition
B (proposed)	42	36 eligible; 5 crashed before the release command (three in the $0.30 \text{ kg}/4 \text{ ms}^{-1}$ figure-eight, two during lift at 2 ms^{-1}); 1 did not physically attach
A (command-triggered)	42	25 eligible; 17 pre-release crashes, 11 of them during or just after lift at $0.30 \text{ kg}/4 \text{ ms}^{-1}$

Table 5.5 uses the 19 pairs with both arms eligible; Table 5.3 uses all 36 eligible arm B flights; and Table 5.4 uses the 32 that completed. Tail-window tracking is not compared, because arm B brakes to hover after an unresolved timeout while arm A continues the figure-eight.

5.13 Development-Stage Evidence

Two earlier tests motivated the final design but used earlier code and do not validate the deployed detector. They are reported because the design decisions they produced are load-bearing in Chapter 4.

First, the quantity-residual fast branch without moment evidence declared release prematurely in 3 of 9 randomised flights during a 4 ms^{-1} figure-eight, the earliest 51.1s before the command. This is the observation that made moment evidence mandatory in (4.16).

Second, after that branch was removed, a 12-flight randomised test of the moment-gated detector — run before the frozen reference of Section 4.4.4 was introduced — produced no pre-command declaration, 11 moment-gated releases with a median detector delay of 1.9s, and one flight that remained UNRESOLVED.

Chapter 6

Conclusion and Future Work

6.1 Summary

This dissertation has treated the quadrotor payload problem as one coupled estimation-and-control task with a specific structural difficulty at its centre. The predictive model changes at grasp and at release in three ways: mass changes the thrust-to-acceleration gain and the hover thrust, attachment geometry increases roll and pitch inertia, and the horizontal centre-of-mass offset creates a standing trim moment. Treating these as one undifferentiated disturbance obscures that they act through different physical channels and are identifiable to different degrees.

The structural difficulty is that the natural parameterisation of those changes — a payload mass together with a grasp offset — is singular at release. Chapter 3 showed that as $m_P \rightarrow 0$ the offset $\mathbf{r}_{xy}$ becomes undefined, that holding it fixed leaves a phantom trim moment in the model, and that the only way an optimiser can cancel that phantom is by corrupting the mass estimate. The remedy adopted is to estimate the horizontal *first mass moment* $\mathbf{s} = m_P \mathbf{r}_{xy}$ — the product the equations of motion actually contain — alongside the composite mass. From that pair the centre-of-mass offset follows as $\mathbf{c}_{xy} = \mathbf{s}/m_T$ with no division by the payload mass and no knowledge of the grasp geometry, the off-diagonal inertia follows with the payload mass cancelling exactly,

and the dominant roll/pitch increment follows from the composite mass and a weak vertical-arm prior whose error was measured to be immaterial.

Two further reductions were necessary and were quantified rather than assumed. The second-order horizontal inertia terms carry a factor $1/m_P$ in the new coordinates and were discarded at a cost of 2.6% of the increment. The remaining mass-dependent coefficient was shown to give the optimiser a lever by which raising the mass estimate dilutes a phantom moment — a mechanism that drove the estimate 53% high in replay — and was frozen within each window to break that derivative while still tracking the true mass between windows.

Around this model, Chapter 4 built a 16-state moving horizon estimator driven by a wrench reconstructed from rotor speeds, a single atomic and health-gated interface carrying that estimate to the controller, and a lifecycle in which *no attach or release notification is evidence*. The controller issues the release command itself and does not consult it. The model is cleared only by persistent estimator confidence; a release that cannot be confirmed becomes UNRESOLVED, a hold in which the vehicle brakes smoothly to hover and keeps waiting, rather than an assumption that the payload has gone.

In software-in-the-loop, Chapter 5 reported that the complete interface confirmed release in 32 of 36 eligible flights through three distinct confirmation paths, with median delays of 3.46s in manoeuvre and 16.56s after braking to hover. Under a 4s delayed detachment, persistent confirmation avoided the early model clearing of a command-triggered baseline in 5 of 5 pairs. The eventless interface additionally detected 10 of 18 unsignaled payload losses that a command-armed confirmer cannot see, at the cost of a detector exposed throughout loaded flight — a cost that produced no false declaration in 209 loaded flights.

6.2 Principal Findings

1. **The first mass moment, not the offset, is the estimable payload geometry.**
The equations of motion contain only the product. Estimating the factors separately adds an unidentifiable direction and a singularity at exactly the operating point of interest.
2. **Payload *presence* must not be encoded in the mass channel.** When it is, the only way for the estimator to express “the load has gone” is to move the mass, which is the one quantity the controller most needs to be correct. Augmenting s gives presence its own channel.
3. **A parameter that appears in a denominator will be abused by an optimiser.** Both pathologies found in this work — the $1/m_P$ coefficient of the discarded inertia terms, and the mass in the denominator of the ghost-moment expression — are instances of the same phenomenon. The fix in both cases was structural, not a matter of tuning.
4. **An adaptive loop must be driven by a signal that is not a product of the adaptation.** Feeding the controller’s intended thrust to the estimator makes the mass unobservable by construction; the rotor-speed reconstruction breaks the circularity, and extending it to the yaw-reaction channel removed the last place where the estimator could learn the controller’s intent.
5. **Release evidence must come from the rotational channel.** A collective-thrust residual cannot distinguish a payload change from a trajectory change or a manoeuvre; a rule that omitted mandatory moment evidence released prematurely in 3 of 9 flights, up to 51 s early. Making moment collapse mandatory costs misses, which are handled as UNRESOLVED holds rather than by lowering the evidence bar.
6. **A ratio needs a reference, and the reference must be formed autonomously and then frozen.** A running maximum absorbs the very estimation errors

the detector exists to reject.

7. **Timeout is not proof.** Treating an unconfirmed release as complete reintroduces the failure the design exists to prevent. UNRESOLVED is the state that makes eventlessness deployable.
8. **Payload adaptation must update the objective as well as the state equation.** A stale equilibrium-input baseline produces a steady-state error that gets *worse* as the input weight is increased — a signature that identifies the fault and that no weight tuning can remove.
9. **The interface is severely sensitive to thrust-map calibration.** A $\pm 5\%$ error in the estimator's thrust coefficient defeated payload recognition in both directions. This is the principal barrier to hardware transfer, and it is a direct consequence of recovering an absolute mass from an absolute thrust.

6.3 Limitations

All system-level evidence is software-in-the-loop, and the simulator's plant generates thrust with the same square law the estimator inverts. Rotor speeds are exact; speed noise, transport delay and induced inflow are untested.

The attachment is rigid. Gripper compliance, pendulum modes, flexible and sloshing loads are not modelled. A sustained vertical gust remains intrinsically confounded with a payload change on a collective-thrust residual, and wind is not evaluated.

Within the interface itself, the dominant failure is not a wrong decision but an unavailable one: in 13 of 32 completed flights the frozen moment reference never formed, so the ratio test never ran, and those flights are also the population of seven of the eight unsignaled-loss misses. Out-of-envelope first-moment samples are excluded from votes but still reach the controller's centre-of-mass

parameter through filtering. The release command is not gated on estimator health, and one divergence is directly attributable to that omission.

The heaviest condition, 0.30 kg at 4 ms^{-1} , sits at the vehicle's torque limit rather than at an operating point: pitch torque saturated within 98 % of diagnostic windows there. Safe release *semantics* are claimed at that limit; reliable flight is not.

Finally, the baseline set is thin. The reported comparisons isolate release semantics only. No comparison against a fixed-parameter controller, a lumped-disturbance adaptive controller, or a formal change-detection procedure under a matched false-alarm constraint is included, although implementations of the first two exist in the codebase.

6.4 Future Work

Reference formation. The single highest-value improvement is to make the loaded moment reference form reliably. Roughly 40 % of flights never produced one, and that single deficiency accounts for the brake-to-hover delay mode, most unsignaled-loss misses, and the confidence-only confirmations. Candidates include forming the reference during the lift transient rather than requiring a later quiet window, admitting a direction-only reference when the magnitude is unstable, and exiting the manoeuvre early on the envelope-exceedance counter that already correlates with the outcome at 4 ms^{-1} .

Calibration robustness. Because the mass channel consumes an absolute thrust, its accuracy is bounded by the accuracy of k_f . Two directions are open: propagating a calibration uncertainty into the estimator weights so that the interface degrades gracefully rather than at a threshold, and building an independent cross-check out of a *calibration-invariant* quantity. One such

quantity is already available on the platform: in steady flight the horizontal centre-of-mass offset is fixed by the *ratio* of the rotor speeds, in which k_f cancels exactly, so it can be formed without trusting the thrust map at all. It would have to be gated to quiet flight and filtered, which is why it is a cross-check on the mass channel rather than a replacement for it.

Health-gated release. The release command should not be issued while the estimator is unhealthy. This is a small change with a demonstrated failure behind it.

Baselines and detection theory. A matched-false-alarm comparison against CUSUM and related procedures on the same signals would establish whether the hand-built persistence rules are near the achievable operating curve or far from it. A comparison against a fixed-parameter and an $\mathcal{L}_1$ -adaptive controller would separate the benefit of the payload model from the benefit of adaptation in general.

Modelling extensions. The yaw inertia increment and the second-order horizontal terms could be restored if a formulation is found that avoids the $1/m_p$ degeneracy — for example by regularising the payload mass away from zero only inside those terms. A random-walk treatment of the payload states would remove the remaining one-window transient, provided the Hessian scaling problem it caused can be solved.

Hardware transfer. Transfer should proceed in stages: motor-stand calibration of k_f and k_m against measured thrust and reaction torque; tethered hover with the estimator running open-loop; centred payload pickup; eccentric pickup; and only then dynamic flight. Simulated rotor speeds must be replaced by electronic-speed-controller telemetry, and the noise, delay and quantisation of

that telemetry re-measured rather than assumed. A supervisor monitoring solver status, estimator health and input margin should gate every stage.

6.5 Closing Remark

The lesson that generalises beyond this vehicle is about the relationship between an estimate and a decision. Online estimation is useful only when the estimated quantity enters the controller in a physically and numerically consistent way — and a system that acts on events it has been *told* about, rather than on what it can *observe*, has no way to notice when the two disagree. Choosing a parameterisation that stays well posed through the transition, and refusing to let a command stand in for evidence, are the two decisions that made the rest of this architecture work.

References

- [1] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017.
- [2] David Q. Mayne. Model predictive control: Recent developments and future promise. *Automatica*, 50(12):2967–2986, 2014.
- [3] Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control: Theory and Algorithms*. Springer, 2 edition, 2017.
- [4] Mina Kamel, Michael Burri, and Roland Siegwart. Linear vs nonlinear MPC for trajectory tracking applied to rotary wing micro aerial vehicles. *IFAC-PapersOnLine*, 50(1):3463–3469, 2017.
- [5] Christopher V. Rao, James B. Rawlings, and David Q. Mayne. Constrained state estimation for nonlinear discrete-time systems: Stability and moving horizon approximations. *IEEE Transactions on Automatic Control*, 48(2):246–258, 2003.
- [6] Moritz Diehl, Hans Georg Bock, and Johannes P. Schlöder. A real-time iteration scheme for nonlinear optimization in optimal feedback control. *SIAM Journal on Control and Optimization*, 43(5):1714–1736, 2005.
- [7] Joel A. E. Andersson, Joris Gillis, Greg Horn, James B. Rawlings, and Moritz Diehl. CasADi: A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019.

-
- [8] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados—a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1):147–183, 2022.
- [9] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados: A modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 14(1):147–183, 2022.
- [10] Gianluca Frison and Moritz Diehl. HPIPM: a high-performance quadratic programming framework for model predictive control. In *IFAC-PapersOnLine (21st IFAC World Congress)*, volume 53, pages 6563–6569, 2020.
- [11] D. Mellinger, Q. Lindsey, M. Shomin, and V. Kumar. Design, modeling, estimation and control for aerial grasping and manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2668–2673, 2011.
- [12] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). In *IEEE Conference on Decision and Control (CDC)*, pages 5420–5425, 2010.
- [13] Philipp Foehn, Angel Romero, and Davide Scaramuzza. Time-optimal planning for quadrotor waypoint flight. *Science Robotics*, 6(56):eabh1221, 2021.
- [14] Anil Aswani, Humberto Gonzalez, S. Shankar Sastry, and Claire Tomlin. Provably safe and robust learning-based model predictive control. *Automatica*, 49(5):1216–1226, 2013.
- [15] Lukas Hewing, Kim P. Wabersich, Melanie Menner, and Melanie N. Zeilinger. Learning-based model predictive control: Toward safe learning in

- control. *Annual Review of Control, Robotics, and Autonomous Systems*, 3:269–296, 2020.
- [16] Gonzalo Torrente, Elia Kaufmann, Philipp Föhn, and Davide Scaramuzza. Data-driven MPC for quadrotors. *IEEE Robotics and Automation Letters*, 6(2):3769–3776, 2021.
- [17] Guillem Torrente, Elia Kaufmann, Philipp Foehn, and Davide Scaramuzza. Data-driven MPC for quadrotors. *IEEE Robotics and Automation Letters*, 6(2):3769–3776, 2021.
- [18] Tim Salzmann, Elia Kaufmann, Juan Arrizabalaga, Marco Pavone, Davide Scaramuzza, and Markus Ryll. Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms. *IEEE Robotics and Automation Letters*, 8(4):2397–2404, 2023.
- [19] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5:411–444, 2022.
- [20] Drew Hanover, Philipp Foehn, Sihao Sun, Elia Kaufmann, and Davide Scaramuzza. Performance, precision, and payloads: Adaptive nonlinear MPC for quadrotors. *IEEE Robotics and Automation Letters*, 7(2):690–697, 2022.
- [21] Ewoud J. J. Smeur, Qiping Chu, and Guido C. H. E. de Croon. Adaptive incremental nonlinear dynamic inversion for attitude control of micro air vehicles. *Journal of Guidance, Control, and Dynamics*, 39(3):450–461, 2016.
- [22] Christopher V. Rao, James B. Rawlings, and David Q. Mayne. Constrained state estimation for nonlinear discrete-time systems: stability and moving horizon approximations. *IEEE Transactions on Automatic Control*, 48(2):246–258, 2003.

-
- [23] Bingheng Wang, Zhengtian Ma, Shupeng Lai, and Lin Zhao. Neural moving horizon estimation for robust flight control. *IEEE Transactions on Robotics*, 40:639–659, 2024. open source: RCL-NUS/NeuroMHE.
- [24] C. G. Atkeson, C. H. An, and J. M. Hollerbach. Estimation of inertial parameters of manipulator loads and links. *The International Journal of Robotics Research*, 5(3):101–119, 1986.
- [25] Teodor Tomić, Christian Ott, and Sami Haddadin. External wrench estimation, collision detection, and reflex reaction for flying robots. *IEEE Transactions on Robotics*, 33(6):1467–1482, 2017.
- [26] James Svacha, James Paulos, Giuseppe Loianno, and Vijay Kumar. IMU-based inertia estimation for a quadrotor using Newton–Euler dynamics. *IEEE Robotics and Automation Letters*, 5(3):3861–3867, 2020.
- [27] Valentin Wüest, Vijay Kumar, and Giuseppe Loianno. Online estimation of geometric and inertia parameters for multirotor aerial vehicles. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1884–1890, 2019.
- [28] Christian Böhm, Martin Scheiber, and Stephan Weiss. Filter-based online system-parameter estimation for multicopter UAVs. In *Robotics: Science and Systems (RSS)*, 2021. paper 087, DOI 10.15607/RSS.2021.XVII.087.
- [29] Koushil Sreenath, Nathan Michael, and Vijay Kumar. Trajectory generation and control of a quadrotor with a cable-suspended load — a differentially-flat hybrid system. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4888–4895, 2013.
- [30] Justin Thomas, Giuseppe Loianno, Joseph Polin, Koushil Sreenath, and Vijay Kumar. Toward autonomous avian-inspired grasping for micro aerial vehicles. *Bioinspiration & Biomimetics*, 9(2):025010, 2014.

- [31] Anibal Ollero, Marco Tognon, Alejandro Suarez, Dongjun Lee, and Antonio Franchi. Past, present, and future of aerial robotic manipulators. *IEEE Transactions on Robotics*, 38(1):626–645, 2022.
- [32] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice-Hall, 1993.
- [33] W. P. M. H. Heemels, Karl Henrik Johansson, and Paulo Tabuada. An introduction to event-triggered and self-triggered control. In *IEEE Conference on Decision and Control (CDC)*, pages 3270–3285, 2012.
- [34] Moses Bangura and Robert Mahony. Nonlinear dynamic modeling for high performance control of a quadrotor. In *Australasian Conference on Robotics and Automation (ACRA)*, 2012.
- [35] Lorenz Meier, Dominik Honegger, and Marc Pollefeys. PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6235–6240, 2015.
- [36] Nathan Koenig and Andrew Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 3, pages 2149–2154, 2004.

Appendix A

Implemented Optimal Control Problem Configuration

Table A.1: NMPC configuration (deployed default).

State dimension	13
Input dimension	4
Runtime parameter dimension	23
Prediction stages	$N = 20$
Sampling interval	$\Delta t = 0.05$ s
Prediction horizon	1.0 s
Solve rate	20 Hz; setpoints published at 50 Hz
Integrator	Explicit RK4
NLP method	Full SQP with merit backtracking
QP solver	Partial-condensing HPIPM
Hessian	Gauss–Newton
Thrust box	[1.27 N, 31.35 N] (actuator envelope)
Torque box	± 0.5 Nm roll/pitch, ± 0.2 Nm yaw
Bryson tolerances (state)	0.1 m, 0.3 m/s, 0.05, 0.3 rad/s
Bryson tolerances (input)	2.0 N, 0.5 N m, 0.2 N m
Terminal scaling	$20\times$ position, $1\times$ velocity, $2\times$ attitude, $0.2\times$ rate

Table A.2: MHE configuration (deployed default). The prior-free column lists the values taken by the ablation of Section 4.2.2.

	Deployed	Prior-free ablation
Augmented state dimension	16 (x, m_T, s_x, s_y)	same
Process-noise dimension	13 (physical states only)	same
Runtime parameter dimension	7 (wrench, previous $\hat{m}, r_z$)	same
Estimation stages / interval	$N = 20 / 0.1$ s	same
Moving horizon	2.0 s	same
Mass box	$[0.95 m_B, 5.0 \text{ kg}]$	$[0.2 \text{ kg}, 5.0 \text{ kg}]$
Mass arrival σ	3.16 kg	1000 kg
First-moment box	$ s_i \leq 1.5 \text{ kg m}$	same
First-moment arrival σ	0.1 kg m	same
Initial iterate	window force balance (3.45)	same
Inertia coefficient A	lagged, (3.34); switch value frozen	same
Vertical-arm prior r_z	-0.47 m	same
Stage weighting β_j	$\equiv 1$ (event logic retired)	same
Re-anchor trigger	5 consecutive solver failures	same
Output filter	first order, $\tau = 0.25$ s	same
NLP / QP method	Full SQP / partial-condensing HPIPM	same

Table A.3: Payload interface and lifecycle settings.

Frame fields	$m_T, s, c_{xy}, \Delta J_{\text{diag}}, J_{\text{diag}}, \gamma, \text{health, solution age}$
Input freshness / solution freshness	0.30 s / 0.30 s
Consumer receive-age limit	0.35 s
Controller-side filter τ	0.20 s
Slew limits ($m_T / s / \Delta J$)	0.60 kg/s / 0.080 kg m/s / 0.080 kg m ² /s
Confidence thresholds ($m_P, \ s\ , \Delta J_{xy}$)	(0.015, 0.060), (0.0015, 0.0060), (0.0020, 0.0100)
Confirmation	$\gamma > 0.90$ held 1.0 s
Moment-collapse ratio / sanity bound	$\rho < 0.30 / \ s\ < 0.008 \text{ kg m}$
Quantity-empty threshold / persistence	$m_P < 0.030 \text{ kg} / 5 \text{ frames (slow path)}$
Residual half-window / floor / persistence	0.3 s / 0.9 N / 2 (normal), 4 (strong)
Residual evidence validity	3.0 s
Reference window	20 frames; $\ \omega\ \leq 0.15, \ v_{xy}\ \leq 0.20$
Reference admissibility	$\ s\ \leq 0.039 \text{ kg m}$
Reference statistics	$\text{CV} \leq 0.25$; direction ≥ 0.90 ; 20 %-trimmed mean
UNRESOLVED timeout / brake duration	12 s / 3 s
Body-rate scale cap / dead band	5.0 / 0.08
Body-rate scale τ / rise / recover	0.5 s / 4.0 s^{-1} / 1.5 s^{-1}
Body-rate headroom fraction / hard limit	0.95 / 2.0 rad s^{-1}
Attach inertia bootstrap	envelope 0.30 kg; confirm 0.5 s; fade 0.5 s

Appendix B

Experiment Record Template

Each run stores the following metadata with its log, so that any reported figure can be regenerated from retained data:

1. repository revision, build timestamp and generated-solver identity;
2. the *resolved value* of every environment switch that affects behaviour — not only those explicitly set, since a default that is never written to disk is the commonest way for a batch to become unattributable;
3. airframe mass and inertia calibration, rotor coefficients and the fixed actuator limits;
4. payload mass and attachment geometry for offline scoring, together with evidence that neither reached the adaptive path;
5. trajectory parameters, grasp and release phase, and random seed;
6. both horizons, weights, bounds and solver options;
7. topic rates and measured end-to-end latency;
8. the complete lifecycle event trace, with timestamps for every state transition and every release vote;
9. termination reason, solver status sequence and constraint activity.

Results tables carry one row per run in the retained data, not only an aggregate, so that failed runs cannot disappear during manual selection. Runs terminated

by numerical failure remain in the denominator; any manual exclusion must name a pre-declared logging or infrastructure fault.

Appendix C

Pre-Submission Checklist

1. Replace the candidate, programme, supervisor and acknowledgement placeholders in `main.tex` and the front matter.
2. Confirm the declaration statements against the current NTU MAE wording.
3. Set `\showdraftnotesfalse` in `main.tex`.
4. Verify every numerical claim in Chapter 5 against a retained log or analysis script, and confirm that no reported batch mixes configurations.
5. Confirm that no run in any reported batch passes payload truth, a task-specific payload mass, or an attach/release notification into the adaptive path, and that the platform envelope is the only mass-valued constant remaining.
6. State, for each reported arm, which release baseline setting was active.
7. Run BibTeX and recompile until all cross-references resolve.
8. Check the final PDF against the current NTU MAE formatting checklist.